UNIDAD 4

PROGRAMACIÓN MODULAR, CLASES Y OBJETOS

1. PROGRAMACIÓN MODULAR Y CLASES.
2. SUBRUTINAS ESTÁTICAS.
   2.1. Definir subrutinas.
   2.2. Llamando subrutinas.
   2.3. Variables Miembro: Globales y Locales.
3. PARÁMETROS.
   3.1. Usando parámetros.
   3.2. Parámetros formales y actuales.
   3.3. Sobrecarga (Overload) de Métodos.
   3.4. Parámetros Arrays.
   3.5. Parámetros desde Línea de Comandos.
   3.6. Número Variable de Parámetros.
4. PRECONDICIONES, POSTCONDICIONES Y ASERCIONES.
   4.1. Disparar Excepciones (throw).
   4.2. Aserciones (Assert).
5. DEVOLVER VALORES (return).
6. RECURSIVIDAD.
7. API'S Y PACKAGES.
8. OBJETOS.
   8.1. Métodos y Variables de Instancia.
   8.2. Diferencias entre Objeto y Clase.
   8.3. Getters y Setters.
   8.4. Objetos Arrays y Strings.
9. CONSTRUCTORES E INICIALIZACIÓN.
   9.1. Inicializar Variables.
   8.2. Constructores.
   8.3. Recolección de Basura.
10. USAR OBJETOS EN PROGRAMAS.
   10.1. La clase Object.
   10.2. Ejemplo de Como Escribir Clases.
11. EJERCICIOS.

BIBLIOGRAFÍA:

- **Java,Cómo Programar (10ª Ed.) Pearson. Paul Deitel y Harvey Deitel (2016).**
- **Introduction to Programming Using Java (7ª Ed.). David J. Eck (2014)**

4.1. PROGRAMACIÓN MODULAR Y CLASES.

La experiencia ha demostrado que la mejor manera de desarrollar y mantener un programa extenso es construirlo a partir de pequeñas piezas sencillas o **módulos**. Esta técnica se llama **divide y vencerás**.

Para escribir programas en Java, se combinan los nuevos métodos y clases que el programador define, con los métodos y clases predefinidos que están disponibles en la **Interfaz de Programación de Aplicaciones de Java** (también conocida como la **API de Java** o **biblioteca de clases de Java**) y en otras librerías de clases. Por lo general, las clases relacionadas están agrupadas en ***paquetes***, de manera que se pueden *importar* a los programas y *reutilizarse*.

La API de Java proporciona una vasta colección de clases predefinidas que contienen métodos para realizar cálculos matemáticos comunes, manipulaciones de cadenas, manipulaciones de caracteres, operaciones de entrada/salida, operaciones de bases de datos, operaciones de red, procesamiento de archivos, comprobación de errores y más...

> ***Nota***: *cuanto más familiarizado estés con la API de Java (para consultarla: http://docs.oracle.com/javase/12/docs/api/) más fácilmente sacarás adelante tus programas, porque mucho del trabajo que tengas que realizar ya lo tienes programado => No reinventar la rueda implica reducir el tiempo de desarrollo y evita que se introduzcan errores de programación.*

Las clases y sus métodos nos ayudan a dividir un programa en módulos,

por medio de la separación de sus tareas en unidades autónomas. Las instrucciones en los cuerpos de los métodos se escriben sólo una vez, se ocultan de otros métodos y se pueden reutilizar en otros programas.

Una razón para dividir un programa en módulos usando los métodos es el enfoque *divide y vencerás*, que **hace que el desarrollo de programas sea más fácil**, ya que se pueden construir programas a partir de piezas pequeñas y simples. Otra razón es la **reutilización de software** (al usar los métodos existentes como bloques de construcción para crear nuevos programas). A menudo se pueden crear programas a partir de métodos estandarizados, en vez de tener que crear código personalizado. Por ejemplo, en los programas anteriores no tuvimos que definir cómo leer datos del teclado, Java proporciona estas herramientas en la clase **Scanner**. Una tercera razón es ***evitar la repetición de código***. El proceso de dividir un programa en métodos significativos hace que el programa sea más fácil de depurar y mantener.

> ***Nota***: *Para conseguir la reutilización de software, cada método debe limitarse a realizar una sola tarea bien definida, y su nombre debe expresar esa tarea con efectividad. El motivo es que un método pequeño que lleva a cabo una tarea es más fácil de probar y depurar que uno más grande que realiza muchas tareas. Pero tampoco descomponga sin motivo una tarea.*
>
> ***Nota***: *si no puede elegir un nombre conciso que exprese la tarea de un método, tal vez esté tratando de realizar diversas tareas en un mismo método. Por lo general, es mejor dividirlo en varias declaraciones de métodos más pequeños.*

Un método se invoca mediante una llamada y cuando el método que se llamó completa su tarea, devuelve el control, y posiblemente un resultado, al método que lo llamó. Una analogía a esta estructura de programa es la forma jerárquica de la administración (figura 1). Un jefe (el solicitante) pide a un trabajador (el método llamado) que realice una tarea y que le reporte (devuelva) los resultados después de completar la tarea. El método jefe no sabe cómo el método trabajador realiza sus tareas. Tal vez el trabajador llame a otros métodos trabajadores, sin que lo sepa el jefe. Este "ocultamiento" de los detalles de implementación fomenta la buena ingeniería de software.

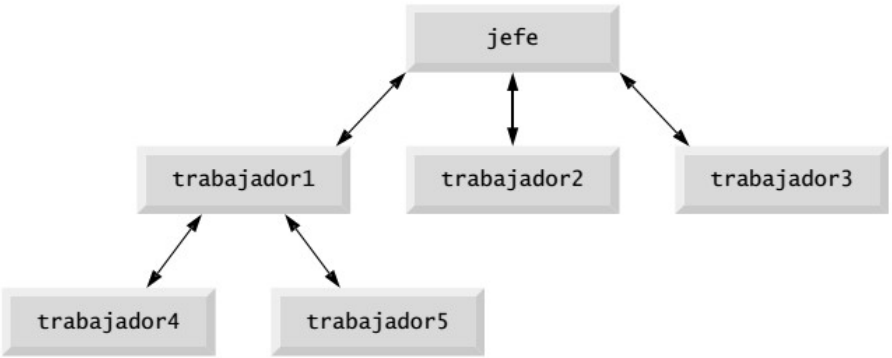


Figura 1: Cadena de llamadas (analogía con métodos de un programa)

Cuando se llama a un método que devuelve un valor que indique si el método realizó correctamente su tarea, debe comprobarse el valor de retorno de ese método y si no tuvo éxito, lidiar con el problema de manera apropiada.

SUBRUTINAS O MÉTODOS

Es una forma de romper la complejidad de un programa en piezas más manejables. Una subrutina es un grupo de sentencias agrupadas, a las

que se da un nombre y que al ejecutarse realizan una tarea. El nombre se usa cada vez que queramos realizar esa tarea en un programa.

Las subrutinas pueden llamarse una y otra vez desde cualquier parte de un programa (una subrutina puede llamar a otras subrutinas). En Java las subrutinas pueden ser **estáticas** o **no estáticas**.

PRINCIPIO DE CAJA NEGRA

Las subrutinas son la principal herramienta que permite vencer la complejidad de un problema (no la única, una clase también usa el mismo principio). Se dice que es una caja negra porque muchas veces no sabes lo que hay dentro de ella (o mejor dicho, no quieres saberlo), no te interesa ahondar en cómo realiza el trabajo que hace, solo quieres aprovecharte de que lo hace.

Pero una caja negra que no es capaz de interactuar con el mundo, no es útil. Por eso las subrutinas tienen una interfaz, un mecanismo que les permite recibir datos del mundo exterior a ellas y también devolver datos a quien las llame. Esta interfaz debe seguir unas reglas:

- Bien definida y fácil de comprender: ejemplos del mundo real pueden ser electrodomésticos, puedes manejar una TV (encenderla, cambiar de canal, cambiar volumen...) sin necesidad de hacer un curso de postgrado en electrónica.
- Permanente: si cambia el interior, pero la interfaz no cambia, los programas pueden seguir funcionando sin cambios.

La interfaz de una subrutina tiene una semántica y una sintaxis que hay que conocer. Para usarlas, necesitas comprender el trabajo que realizan (sin saber como lo hacen), qué datos hay que pasarle, qué resultados te va a devolver y como se escribe (sintaxis) esa petición.

El contrato de una subrutina: Esto es lo que tienes que hacer para

usarme y este es el trabajo que yo te haré... Cuando no tengas subrutinas que hagan lo que quieres, podrás hacer nuevas.

4.2 SUBRUTINAS Y VARIABLES ESTÁTICAS.

En Java, una subrutina debe definirse dentro de una clase (la mayoría de lenguajes de programación permiten definir subrutinas independientes). Así que en Java, una de las utilidades de las clases es **agrupar datos y subrutinas relacionadas**. Las clases a su vez, se agrupan en paquetes.

En una clase de Java pueden definirse **subrutinas estáticas** (pertenecen a la clase) y **no estáticas** (pertenecen a cada objeto que se cree a partir de esa clase). De hecho, cualquier cosa que se define en una clase puede ser estática o no. En **POO** (**Programación Orientada a Objetos**) a una subrutina de una clase u objeto se le llama **método**.

4.2.1 DEFINICIONES DE SUBRUTINAS.

La definición incluye el nombre y el código que contiene (las sentencias) y que se ejecutará cada vez que se llame desde un programa. Sintaxis:

```
modificadores tipo_devuelto nombre ( lista_parámetros ) {
  sentencias;
}
```

Por ejemplo, la subrutina **main()** de los programas que ya has creado hasta ahora te servirá como muestra. El bloque de sentencias que se ejecuta es el cuerpo de la subrutina, la implementación o el detalle de la caja negra.

Los modificadores son palabras reservadas que definen ciertas características que va a tener. Los que verás con mucha frecuencia son

static que indica que es estática (de la clase) y **public**. Aunque en realidad hay casi media docena. El significado de algunos:

- **public**: el método puede llamarse (la variable puede utilizarse) desde cualquier parte de un programa.
- **protected**: es accesible desde la propia clase, desde las clases hijas de la clase y desde el paquete donde está la clase.
- **Sin indicar**: al no indicar ningún modificador de acceso, usa el acceso por defecto (package) y el elemento solo puede ser accedido desde la clase y desde cualquier clase en el paquete que la contiene.
- **private**: el método puede llamarse (la variable puede utilizarse) solamente desde dentro de la misma clase.

Si la subrutina es una función (devuelve un valor), entonces el valor que devuelve debe ser del tipo de dato de la definición, como **String**, **int** o un array como **double[]**, o un objeto de alguna clase.

Si la subrutina no devuelve ningún valor (es un procedimiento), el tipo de dato que se indica será **void** (vacío o no existente).

Por último, **el nombre y la lista de parámetros son lo que define la interfaz de la subrutina**. Representan la información que se le pasa a la subrutina para que pueda hacer su trabajo. Por ejemplo, imagina que le dices a una TV que cambias de canal ¿Le dices a qué canal cambiar? Bueno, pues esa información es un parámetro. Una subrutina puede tener cero o más parámetros separados por comas. Cada uno tiene un tipo de dato y un nombre por el que se conoce en el código de la subrutina. Ejemplo:

```
public void cambiaCanal( int numeroCanal ) { ... }
```

Cuando se quiera utilizar en un programa, la llamada será:

```
cambiaCanal(5);
```

Cuando tengas varios parámetros, aunque sean del mismo tipo, debes indicarlo para cada uno, es decir no vale escribir **(double x,y)** si no que hay que indicar el tipo para cada uno: **(double x, double y)**.

Ejemplos de definiciones de subrutinas:

```
public static void jugarPartida() {
   // "public" y "static" son modificadores
   // "void" es lo que devuelve (nada -> es un procedure)
   // jugarPartida es el nombre de la subrutina
   // La lista de parámetros está vacía
   // Las declaraciones y sentencias van aquí...
}

int dameSiguienteN(int n) {
   // No hay modificadores
   // devuelve un "int" -> es una función
   // "dameSiguienteN" es el nombre
   // Parámetros: tiene un parámetro de tipo int y nombre n
   // Las sentencias van aquí ...
}

static boolean menorQue(double x, double y) {
   // Modificadores: "static" -> es de la clase
   // Devuelve: boolean -> es una función
   // Parámetros: dos de tipo double que se llaman x e y
   // Las sentencias van aquí
}
```

EJERCICIO 1: Completa la descripción de esta subrutina main():

public static void main(**String**[] args) { ... }
// Modificadores: _____
// Devuelve: _____
// Parámetros: _____
// Nombre de la subrutina: _____

4.2.2 LLAMANDO A LAS SUBRUTINAS.

Al definir una subrutina, indicas que existe y lo que hace, pero no se ejecuta hasta que se realice una llamada (incluso es cierto para main(), que es llamada por el sistema operativo).

La llamada a **jugarPartida()** puede ocurrir en la misma clase donde se define, en un método **main()** o en otra subrutina de otra clase, al ser pública. Pero en caso de llamarla desde otra clase diferente, debes indicar a qué clase pertenece. Si la clase donde se ha definido se llama **Poker**, la llamada será:

```
Poker.jugarPartida();
```

Poner el nombre de la clase delante, ayuda a saber donde está la subrutina, porque podrías tener otra subrutina similar en la clase **Parchis**, por ejemplo. Así que para llamar a subrutinas estáticas, normalmente se usan estas dos sintaxis:

```
nombre_subrutina(parámetros); // Desde su misma clase
nombre_clase.nombre_subrutina(parámetros); // Desde otra clase
```

> ***Nota**: Las subrutinas que no son estáticas, siempre necesitan el objeto que debe ejecutarlas.*

EJEMPLO 1: Vamos a hacer un programa completo que incluya varias subrutinas, incluidas main(). Vamos a programar un juego, de forma que el programa escoge al azar un número entre 1 y 100 y el usuario intenta adivinarlo. Si no lo consigue, el programa le indica si es menor o mayor.

```
package juegos;

import java.util.Scanner;
```

```
public class AdivinaNumero {
   static Scanner teclado;

   public static void main(String[] args) {
      teclado = new Scanner( System.in );
      System.out.println("Vamos a Jugar a que adivines un nº");
      System.out.println("entre 1 y 100.");
      boolean otraVez;
      do {
         juegaPartida(); // Llamada a subrutina
         System.out.print("Juegas otra vez? (S/N)");
         otraVez = teclado.next().equals("S")? true:false;
      } while (otraVez);
      System.out.println("Gracias por Jugar. Xao.");
   } // fin main()

   static void juegaPartida() {
      int numero;    // El nº aleatorio
      int respuesta; // el nº indicado por el usuario
      int intentos;  // nº de intentos

      numero = (int)(100 * Math.random()) + 1; // entre 1 y 100
      intentos = 0;
      System.out.println();
      System.out.print("Cuál es su primer valor? ");
      while (true) {
         respuesta = teclado.getInt(); // leer respuesta
         intentos++;
         if (respuesta == numero) {
            System.out.println("Acertado en " + intentos +
                " intentos! Has ganado. El número era " + numero);
            break; // juego acabado
         }
         if (intentos == 6) {
            System.out.println("Has perdido, límite 6 intentos.");
            System.out.println("El número era " + numero );
            break;
         }
         // Dar pistas
         if (respuesta < numero)
            System.out.print("Demasiado bajo. Prueba otra vez: ");
         else if (respuesta > numero)
            System.out.print("Demasiado alto. Prueba otra vez: ");
```

```
            }
            System.out.println();
        } // fin juegaPartida()

    } // fin clase AdivinaNumero
```

4.2.3 VARIABLES MIEMBRO, LOCALES Y GLOBALES.

Una clase, además de incluir subrutinas, también puede declarar variables. Además de declarar variables dentro de las subrutinas (variables locales), puedes hacerlo dentro de una clase (variables miembro o globales).

Las variables miembro de una clase pueden ser estáticas (de la clase) o no estáticas (de cada objeto de la clase). Una variable estática existe mientras lo haga la clase donde se define. Cualquier valor que se almacene en la variable estática es guardado en la clase. Se puede utilizar desde cualquier subrutina. Es un dato compartido por todas las subrutinas estáticas de la clase.

Una variable local, por contra, solo existe mientras se ejecutan las sentencias de la subrutina. Cuando la subrutina acaba, las variables locales desaparecen y son inaccesibles desde fuera (no existen).

La declaración de las variables miembro es igual que la de una variable local excepto por dos cosas: se hace fuera de una subrutina (aunque dentro de la clase) y puede tener modificadores de acceso (**public**, **private**) y **static**. Ejemplos:

```
static String nombreUsuario;
public static int numeroJugadores;
private static double velocidad, tiempo;
```

Una variable miembro estática que no es privada puede ser accedida desde fuera de la clase donde se declara además de desde dentro.

Cuando se usa desde alguna otra clase hay que indicar el nombre de la clase:

```
clase.variable
```

Vamos a añadir dos variables miembro estáticas a la clase AdivinaNumero:

```
static int partidasJugadas;
static int partidasGanadas;
```

Cuando declaras una variable local en una subrutina, debes darle un valor inicial antes de usarla por primera vez. **A las variables miembro sin embargo, se les asigna un valor por defecto al declararlas**. Los valores numéricos se inicializan a cero, las variables booleanas a false, las variables char con el carácter Unicode número cero, y los objetos como Strings, el valor inicial por defecto es **null**.

```
package juegos;

import java.util.Scanner;

public class AdivinaNumero {
   static Scanner teclado;
   static int partidasJugadas; // nº de partidas jugadas
   static int partidasGanadas; // nº de partidas ganadas

   public static void main(String[] args) {
      teclado = new Scanner( System.in );
      partidasJugadas = 0;
      partidasGanadas = 0;
      System.out.println("Vamos a Jugar a que adivines un nº");
      System.out.println("entre 1 y 100.");
      boolean otraVez;
      do {
         juegaPartida(); // Llamada a subrutina
         System.out.print("Juegas otra vez? (S/N)");
         otraVez = teclado.next().equals("S")? true:false;
      } while (otraVez);
      System.out.println("Has jugado " + partidasJugadas + "
```

```
                                 veces, y has ganado " + partidasGanadas);
            System.out.println("Gracias por Jugar. Xao.");
        } // fin main()

        static void juegaPartida() {
            int numero;     // El nº aleatorio
            int respuesta; // el nº indicado por el usuario
            int intentos;  // nº de intentos

            numero = (int)(100 * Math.random()) + 1; // entre 1 y 100
            intentos = 0;
            System.out.println();
            System.out.print("Cuál es su primer valor? ");
            while (true) {
                respuesta = teclado.getInt(); // leer respuesta
                intentos++;
                if (respuesta == numero) {
                    System.out.println("Acertado en " + intentos +
                        " intentos! Has ganado. El número era " + numero);
                    partidasGanadas++;
                    break; // juego acabado
                }
                if (intentos == 6) {
                    System.out.println("Has perdido, límite 6 intentos.");
                    System.out.println("El número era " + numero );
                    break;
                }
                // Dar pistas
                if (respuesta < numero)
                    System.out.print("Demasiado bajo. Prueba otra vez: ");
                else if (respuesta > numero)
                    System.out.print("Demasiado alto. Prueba otra vez: ");
            }
            System.out.println();
            partidasJugadas++;
         } // fin juegaPartida()

    } // fin clase AdivinaNumero
```

Las clases que no declaran un paquete están en el paquete por defecto, así que cualquier clase que no declare un paquete tendrán acceso a partidasJugadas, partidasGanadas y juegaPartida().

> **Nota**: *Es buena costumbre hacer que las variables miembro y las subrutinas sean privadas, salvo que haya un buen motivo.*

VARIABLES LOCALES.

Dentro de una subrutina puedes usar tres tipos de variables: las variables locales (se declaran en la subrutina), parámetros formales y variables definidas en una clase (y declaradas fuera de la subrutina).

Las variables locales no son visibles ni existen fuera de la subrutina donde se declaran. Solo se utilizan dentro de la subrutina. Los parámetros se usan para enviar valores a la subrutina cuando se llama, una vez que la subrutina comienza su ejecución, son como variables locales. Los cambios que hagan las sentencias de la subrutina a los parámetros, no tienen efecto en el exterior (salvo que el parámetro sea un objeto).

Pero si la variable es definida fuera de la subrutina, los cambios que se realicen si que permanecen cuando la subrutina acaba. Cuando la variable existe independientemente de la subrutina, decimos que es global.

> ***Nota:*** *Es mejor acostumbrarse a minimizar la cantidad de variables globales que una subrutina utilice. Solo cuando sea realmente necesario. Es preferible pasar los valores que necesite a través de parámetros.*

Un buen motivo para usar variables globales es si piensas en una clase como una caja negra también. Es más cómodo que todas las subrutinas de la clase accedan a las variables de la clase, que pasarlas como parámetros.

Cuando se ejecuta la declaración de la variable, se reserva memoria para almacenar su valor. Después de la declaración se realiza una inicialización (una asignación de un primer valor). Ejemplo:

```
int cuenta; // Declara una variable llamada cuenta
cuenta = 0; // se da un valor inicial
```

También es posible hacer las dos cosas en la misma sentencia:

```
int contador = 0; // Declara e inicializa
```

El valor puede ser cualquier expresión compatible con el tipo de dato de la variable declarada, no tiene porqué ser una constante. Además, se pueden declarar e inicializar varias variables en la misma sentencia:

```
char inicial1 = 'D', inicial2 = 'E';
int x, y = 1;          // OK, pero solo se inicializa y!
int N = 3, M = N + 2; // OK, N se inicializa antes de usarla en M
```

En bucles es algo que se utiliza mucho, declarar e inicializar variables que solo se usan una vez, precisamente dentro del bucle:

```
for ( int i = 0; i < 10; i++ ) {
   System.out.println(i);
}
```

Recuerda que eso es una abreviación equivalente a esto:

```
{
   int i;
   for ( i = 0; i < 10; i++ ) {
      System.out.println(i);
   }
}
```

Una variable miembro también puede icicializarse en el momento de decararla:

```
public class Banco {
   private static double ratioInteres = 0.05;
```

```
    private static doubble maxDescubierto = 200.0;
    // resto de variables y subrutinas...
}
```

Una variable estática se crea tan pronto como el Intérprete de Java carga la clase, y la inicialización se produce en ese momento. Esta asignación es la única que puede ocurrir fuera de una subrutina. Por tanto estas sentencias no son legales:

```
public class Banco {
    private static double ratioInteres;
    ratioInteres = 0.05; // ILLEGAL!! FUERA DE SUBRUTINA!!
```

Si a una variable miembro no se le asigna un valor inicial, se utiliza un valor por defecto. Por ejemplo al declarar un entero sin inicializar, se le asigna el valor 0:

```
static int contador;
static int contador = 0;  // Son equivalentes
```

Para inicializar un array al declararlo, debes aportar los valores separados por comas y encerrados entre llaves:

```
// Declarar y rellenar un array de 9 nºs enteros
int[] primerosPrimos = { 2, 3, 5, 7, 11, 13, 17, 23, 29 };
```

La otra forma de inicializar un array, se puede usar tanto al declararlo como después (en una sentencia de asignación), pero los elementos tendrán su valor por defecto:

```
String[] nombres = new String[100];
```

4.3 PARÁMETROS.

Si una subrutina es una caja negra, un parámetro es el mecanismo para intercambiar información desde el mundo exterior hacia la subrutina.

4.3.1 USANDO PARÁMETROS.

Como ejemplo, vamos a realizar la subrutina que implementa el algoritmo que genera la secuencia 3N+1 (si n es impar, el siguiente valor es obtiene multiplicando el actual por 3 y sumando 1, y si es par, el siguiente valor se obtiene dividiéndolo por 2; se continúa mientras el valor no sea 1). Por ejemplo, comenzando por N=3, la secuencia será: 3, 10, 5, 16, 8, 4, 2, 1). La subrutina recibirá el número N donde comienza la secuencia:

```
/**
* Imprime la secuencia 3N+1 por la salida estándar.
* ENTRADAS: el valor inicial de la secuencia. (entero positivo)
*/
static void secuenciaPedrisco(int inicio) {
   int N;        // Un término de la secuencia
   int contador; // El número de términos
   N = inicio;   // El primer término a usar.
   contador = 1; // El primer término
   System.out.println("La secuencia 3N+1 comienza en " + N);
   System.out.println();
   System.out.println(N); // el primer término se imprime
   while (N > 1) {
      if (N % 2 == 1) // si N es impar
         N = 3 * N + 1;
      else
         N = N / 2;
      contador++;    // un nuevo término
      System.out.println(N); // imprime el término
   }
   System.out.println();
   System.out.println("Hay " + contador + " términos.");
} // fin secuenciapedrisco
```

La subrutina tiene un parámetro llamado inicio de tipo entero. Dentro de la subrutina se puede utilizar el valor igual que cualquier otra variable local. Pero el valor que contiene se indica desde el exterior cuando se llame a la subrutina. Por ejemplo, la clase donde esté la subrutina puede contener una subrutina main() que la llame, pasándole

un valor que indique un usuario al programa:

```
package series;

import java.util.Scanner;

public class Pedrisco {

   static Scanner teclado = new Scanner( System.in );

   public static void main(String[] args) {
      System.out.println("Usar la secuencia Pedrisco (3N+1)");
      System.out.println("comenzando en el valor que indiques.");
      System.out.println();
      int K; // Indicado por el usuario
      do {
         System.out.println("Valor inicial (0 para acabar): ");
         K = teclado.nextInt();      // Leer valor
         if (K > 0)
             secuenciaPedrisco(K);
      } while (K > 0); // Continúa solo si el nº K > 0
} // fin main
```

4.3.2 PARÁMETROS FORMALES Y ACTUALES.

La palabra parámetro se utiliza para definir dos conceptos diferentes, pero relacionados. Hay parámetros que se usan cuando se define la subrutina, como el parámetro inicio al definir la subrutina secuenciaPedrisco. Y hay parámetros que se usan en las llamadas a las subrutinas, como k en el código anterior. Los parámetros en la definición de la subrutina son los **parámetros formales**. Los parámetros en las llamadas son **parámetros actuales**.

Un parámetro formal es un identificador y es similar a una variable, así que tiene un tipo de asociado. Un parámetro actual puede ser una expresión del tipo indicado en el parámetro formal o un tipo compatible (por ejemplo si el parámetro formal es double, el actual puede ser un int):

```
static void ejemplo(int N, double x, boolean prueba) {
   // sentencias...
}
```

Una llamada:

```
ejemplo(17, Math.sqrt(z+1), z >= 10);
```

Cuando se ejecute la llamada a la subrutina, el efecto sería como tener un bloque con las sentencias de la subrutina, salvo por las cuestiones del ámbito de las variables:

```
{
int N; // Alojar memoria para los parámetros formales
double x;
boolean prueba;
   N = 17; // rellenar valores de parámetros actuales
   x = Math.sqrt(z+1);
   test = (z >= 10);
   // Sentencias...
}
```

Nota: *Errores comunes con los parámetros de las subrutinas que cometen los programadores novatos y que deberían hacer preguntarse si ese dato es realmente un parámetro o una variable local de la subrutina:*

- ***Inicializar un parámetro al principio de la subrutina*** *-> no es un parámetro*
- ***Pedir al usuario el valor de un parámetro*** *-> no es un parámetro.*

4.3.3 SOBRECARGA DE MÉTODOS.

Para llamar a una subrutina, debes conocer su nombre, cuántos parámetros formales tiene y el tipo de dato de cada uno. Esta información se denomina la **firma de una subrutina**. Ejemplo:

```
ejemplo(int, double, boolean)
```

Observa que la firma no incluye los nombres de los parámetros formales, no son parte de la interfaz de la subrutina.

Java permite que dos subrutinas diferentes definidas la misma clase tengan el mismo nombre, siempre que tengan firmas diferentes. Cuando esto ocurre se dice que **el nombre de la subrutina está sobrecargado** porque tiene diferentes códigos asociados. El intérprete sabe cuál ejecutar, por el tipo de los parámetros actuales que se utilicen en la llamada.

Ya has usado subrutinas sobrecardas cuado has impreso valores, es decir, la subrutina **System.out.println()**:

```
println(int)          println(double)       println(char)
println(boolean)      println()
```

Así que cuando haces una llamada como `System.out.println(17)`, se utiliza `println(int)`, mientras que al hacer la llamada `System.out.println('A')` se utiliza la firma `println(char)`.

> ***Nota***: *el tipo devuelto no está incluido en la firma, no puedes tener dos subrutinas en la misma clase con la misma firma, aunque devuelvan tipos diferentes.*

EJEMPLO 1: Escribir una subrutina que de una clase que imprima por pantalla todos los divisores de un número n.

```
/**
* Imprimir todos los divisores de n
* ENTRADAS: n (entero positivo)
* SALIDAS: ninguna.
*/
static void imprimeDivisores( int n ) {
   int d; // uno de los divisores de n
   System.out.println("Los divisores de " + n + " son:");
```

```
      for ( d = 1; d <= N; d++ ) {
         if ( n % d == 0 )   // d divide a n?
           System.out.println(d);
      }
   }
```

EJEMPLO 2: Escribe una subrutina estática y privada de la clase **Ejem2**, que se llame **rellena** y tenga dos parámetros, uno de tipo carácter y otro de entero e imprima una línea de texto con tantas repeticiones del carácter como valor tenga el parámetro entero. Si el valor es menor o igual de 0, imprime un salto de línea.

```
/**
* Escribe una línea de texto que contiene n copias de la letra c
* ENTRADAS: c y n
* SALIDAS:  ninguna
*/
private static void rellena( char c, int n ) {
   int i;  // variable contador
   for ( i = 1; i <= n; i++ ) {
      System.out.print( c );
   }
   System.out.println();
}
```

EJEMPLO 3: haz una subrutina llamada **rellena25** que acepte de parámetro un `String`. Para cada carácter del `String` debe imprimir una línea con 25 veces ese carácter (usa la subrutina anterior).

```
/**
* For each character in str, write a line of output
* containing 25 copies of that character.
*/
private static void rellena25( String s ) {
   int i;    // variable contador
   for ( i = 0; i < s.length(); i++ ) {
      rellena( s.charAt(i), 25 );
   }
}
```

4.3.4 PARÁMETROS ARRAYS.

Es posible pasar como un único parámetro a una subrutina, un array completo con todos sus valores. Por ejemplo, vamos a hacer una subrutina llamada **muestraListaInt()** a la que pasamos un array a de números enteros y nos los muestra con el formato: "[i_0, i_1, ... i_n]".

```
static void muestraListaInt( int[] a ) {
   System.out.print('[');
   for ( int i = 0; i < list.length; i++ ) {
      if ( i > 0 )
         System.out.print(', '); // el primero sin coma
      System.out.print( a[i] );
   }
   System.out.println(']');
}
```

La llamada a esta subrutina debe usar un parámetro actual de tipo array de enteros, ejemplo:

```
int[] n;
n = new int[3];
n[0] = 42;
n[1] = 17;
n[2] = 256;
muestraListaInt( n );
```

Genera la salida: [42, 17, 256]

4.3.5 ARGUMENTOS POR LÍNEA DE COMANDOS.

La rutina main() de un programa tiene un parámetro de tipo **String[]**. Cuando se llama al programa, hay que pasarle este string de cadenas como parámetro actual. Si se llama desde el SO, este es quien le pasa este array. El array contiene los argumentos u opciones de la línea de comandos cuando se llama al programa. Por ejemplo si hacemos esta llamada desde línea de comandos:

```
c:\java unPrograma   -> No hay ítems en String[] llamado args
```

```
c:\java unPrograma uno dos tres -> args tiene 3 elementos
```

EJEMPLO: Imprime los argumentos de línea de comandos.

```
public class CliDemo {

   public static void main(String[] args) {
      System.out.println("Usted ha tecleado " + args.length +
                  " argumentos");
      if (args.length > 0) {
         System.out.println("Son:");
         for (int i = 0; i < args.length; i++)
            System.out.println(" " + args[i]);
      }
   } // fin main()
} // fin CliDemo
```

En la práctica los argumentos de la línea de comandos se usan frecuentemente para pasar nombre de ficheros por ejemplo. Pero como actualmente la mayoría de programas son gráficos e interactivos, los argumentos de línea de comandos no siempre se usan.

4.3.6 NÚMERO DE PARÁMETROS VARIABLES.

En Java existe la posibilidad de definir métodos que tengan un numero de parámetros variable (indefinido o desconocido). Esta posibilidad del lenguaje se llama VarArgs (argumentos variables) y aparece a partir del J2SE 5.0.

Antes de J2SE 5.0 si necesitabas un método con una cantidad indeterminada de valores, se podía definir con un parámetro array (u otra colección) tal y como se hace en main(). Ejemplo:

```
int sumar(int[] nums) {
    int total = 0;
    for(int i = 0; i < nums.length; i++) total+= nums[i];
    return total;
}
```

para llamarlo, primero se crea un array con los valroes que necesite y

se lo pasa. Ejemplo:

```
int[] array = { 1, 2, 3, 4, 5 };
sumar(array);
```

Si no queremos usar arrays o colecciones podemos sobrecargar el método con varios números de parámetros. Pero es poco flexible y elegante, mejor lo del array:

```
int sumar( int a, int b) { return a + b; }
int sumar( int a, int b, int c) {  return a + b + c; }
int sumar( int a, int b, int c,  int d) {  return a + b + c + d; }
...
```

La nueva sintaxis consiste en añadir al tipo del parámetro variable tres puntos (...). Ejemplo:

```
int sumar(int... nums) {
    int total = 0;
    for(int i = 0; i < nums.length; i++) total+= nums[i];
    return total;
}
```

El tratamiento en el método es como si fuera un array. Pero en el programa que hace la llamada no:

```
sumar(1, 2);
sumar(1, 2, 3, 4, 5);
```

En la llamada nos ahorrarnos tener que crear el array, ya lo hace el comprilador convirtiendo estas llamadas:

sumar(1, 2, 3, 4) en esta: **sumar(new int[] {1, 2, 3, 4})**

<u>Argumentos variables mezclados con otros:</u>
Cuando defines un método, solo puedes tener un parámetro vararg (con argumentos variables) y debe ir al final de la lista de parámetros.
EJEMPLO: Mostrar la tabla de multiplicar de un número n:

```
void tablaMultiplicar(int n, int... v) {
    System.out.println("Tabla de multiplicar de: " + n);
    for (int i=0 ; i < v.length ; i++) {
        System.out.println(v[i] + " x " + n + " = " + (v[i] * n));
    }
}
...
// La llamada podría ser:
tablaMultiplicar(5, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
tablaMultiplicar(3, 0, 1, 2, 3);
```

4.4. PRECONDICIONES Y POSTCONDICIONES.

Los programadores invierten una gran parte de su tiempo en mantener y depurar código. Para facilitar estas tareas y mejorar el diseño en general, se especifican los estados esperados antes y después de la ejecución de un método o subrutina. A estos estados se les llama precondiciones y postcondiciones, respectivamente.

- **Precondiciones**: deben ser verdaderas cuando se invoca a un método. Las precondiciones describen las restricciones en los parámetros de un método, y cualquier otra expectativa que tenga el método en relación con el estado actual de un programa, justo antes de empezar a ejecutarse. Si no se cumplen las precondiciones, entonces el comportamiento del método es indefinido (puede lanzar una excepción, continuar con un valor ilegal o tratar de recuperarse del error). Nunca hay que esperar un comportamiento consistente si no se cumplen las precondiciones.

- **Postcondiciones**: Una postcondición es verdadera una vez que el método regresa con éxito. Las postcondiciones describen las restricciones en el valor de retorno, así como cualquier otro efecto secundario que pueda tener el método.

Al definir un método, **debes documentar todas sus pre y post condiciones.**

Cuando no se cumplen sus precondiciones o postcondiciones, los métodos por lo general lanzan excepciones. Como ejemplo, piensa en el método estático **charAt()** del objeto **String**, que tiene un parámetro int que es un índice que indica la posición que ocupa en el String, el carácter que quieres. Vamos a pensar en las precondiciones y postcondiciones que le vendrían bien. Como precondición, el índice debe ser mayor o igual a 0 y menor que la longitud del String. Si no se cumple se lanza la excepción `IndexOutOfBoundsException`. Postcondiciones no necesita, si se cumple la precondición, el resultado será correcto.

4.4.1 LANZANDO (THROWING) EXCEPCIONES.

El contrato de una subrutina es que el que realice la llamada envíe valores apropiados en los parámetros, pero ¿qué ocurre si el que llama no cumple este compromiso? Lo normal es que la subrutina lance una excepción. Por ejemplo: `Double.parseDouble("Hola");` lanza una excepción porque no estoy pasando en el String un valor convertible a double, lanza `NumberFormatException`.

Podrías querer hacer lo mismo en tus subrutinas. Java tiene la sentencia que te permite lanzar excepciones producidos por valores de parámetros incorrectos, la excepción es de tipo `IllegalArgumentException`:

```
throw new IllegalArgumentException( mensaje_de_erro );
```

EJEMPLO 1: Añadir una precondición en el código del ejercicio secuenciaPedrisco de forma que cuando el número del parámetro inicio sea menor o igual a cero, lance una excepción.

```
static void secuenciaPedrisco(int inicio) {
   if (inicio <= 0) // contrato incumplido!
      throw new IllegalArgumentException( "Inicio debe ser >=0" );
   // sentencias...
```

A veces se confunden las palabras reservadas **throw** y **throws** así que vamos a explicar como se utilizan. Imagina que hacemos un método en el que validamos ciertos datos y si hay alguno sobrepasa cierto valor podemos lanzar una excepción, ya sea creada por nosotros mismos o cualquiera del tipo **java.lang.throwable**, por ejemplo:

```
1  public void pruebaThrow() throws Exception {
2   Exception exception = new Exception();
3   int a = 98;
4   int b = 101;
5
6   if (a > 100 || b > 100) {
7    throw exception;
8   }
9  }
```

En la línea 7 lanzamos la excepción creada en la línea 2. Al hacerlo, si pruebas el programa observarás que la aplicación se detiene.

La palabra reservada **throws** sirve para indicar que un método lanza una excepción de un tipo especifico o general, se puede utilizar si la excepción se va manejar con un try-catch o no. Así se utiliza:

```
1  public static void prueba() throws SQLException {
2   // Manejamos la excepcion de tipo SQL
3  }
4
5  public static void prueba() throws NullPointerException {
6   // Manejamos la excepcion de tipo NULL
7  }
8
9  public static void prueba() throws DataFormatException {
10  // Manejamos la excepcion de tipo DataFormat
11 }
```

Si te fijas en el primer ejemplo de throw ya usamos **throws** para especificar que ese método lanza una excepción.

4.4.2. ASERCIONES.

Condiciones que se deben cumplir en cualquier parte del código. Sirven para asegurar la validez de un programa al atrapar los errores potenciales e identificar posibles errores lógicos durante el desarrollo. Las precondiciones y las poscondiciones son dos tipos de aserciones. Las precondiciones son aserciones sobre el estado de un programa a la hora de invocar un método, y las poscondiciones son aserciones sobre el estado de un programa cuando el método termina.

Aunque las aserciones pueden establecerse como comentarios para guiar al programador durante el desarrollo del programa, Java incluye dos versiones de la instrucción assert para validar aserciones mediante programación. La instrucción **assert** evalúa una expresión boolean y, si es false, lanza una excepción **AssertionError** (una subclase de Error). La primera forma de la instrucción assert es:

```
assert expre; // lanza AssertionError si expre es false
```

La segunda forma es:

```
assert expre1:expre2; /* evalúa expre1 y lanza excepción
AssertionError con expre2 como el mensaje de error, en caso
de que expre1 sea false */
```

Puedes utilizar aserciones para implementar las precondiciones y postcondiciones mediante programación, o para verificar cualquier otro estado intermedio que te ayude a asegurar que el código esté funcionando en forma correcta. Ejemplo:

```
// Comprobar mediante assert que un valor esté dentro del rango.
import java.util.Scanner;

public class PruebaAssert {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);
      System.out.print("Escriba un numero entre 0 y 10: ");
      int n = entrada.nextInt();
      // asegura que el valor sea >= 0 y <= 10
      assert (n >= 0 && n <= 10) : "numero incorrecto: " + n;
      System.out.printf("Usted escribio %d%n", numero) ;
   }
} // fin de la clase PruebaAssert
```

Nota: *Los usuarios no deben encontrar ningún error tipo* ***AssertionError****. Deben usarse sólo durante el desarrollo del programa. Nunca se debe atrapar una excepción tipo AssertionError. Debemos permitir que el programa termine para poder ver el mensaje de error y después hay que localizar y corregir el origen del problema. No debemos usar la instrucción* ***assert*** *para indicar problemas en tiempo de ejecución en el código de producción, debemos usar el mecanismo de las excepciones para eso.*

4.5 VALORES DE RETORNO.

Una subrutina que devuelve un valor es una función. Tienen la característica de que las llamadas a las funciones pueden aparecer en expresiones jugando el papel de cualquier valor. Por ejemplo pueden aparecer en las expresiones de las sentencias de control, en las expresiones de asignación o en las expresiones que calculan el valor de un parámetro a otra subrutina.

Cuando en una subrutina quieres acabar y devolver un valor que sustituye a la llamada en el código, se usa la sentencia return seguida

de una expresión. Sintaxis:

return expressión;

Esta sentencia solamente puede aparecer dentro de una subrutina y el tipo de dato de la expresión debe coincidir con el tipo de dato que se indica en la declaración de la función que se devuelve. Ejemplo:

```
static double pitagoras(double c1, double c2) {
   // Calcula la hipotenusa de un triángulo rectángulo de
   // catetos c1 y c2
   return Math.sqrt( c1 * c1 + c2 * c2 );
}
```

Podemos usar llamadas como esta:

```
total = 17 + pitagoras(12, 5);
```

Cuando la sentencia **return** se ejecuta, la subrutina acaba. Y aunque es habitual que aparezca al final de las subrutinas, realmente puede aparecer en cualquier parte suya.

Ejemplo 1: calcular el siguiente término de la secuencia pedrisco.

```
static int sigTermino( int actual ) {
   if (actual % 2 == 1) return 3 * actual + 1;
   else return actual / 2;
}
```

Ejemplo 2: Una función equivalente con un solo return al final.

```
static int sigTermino(int actual) {
   int respuesta;
   if (actual % 2 == 1)
      respuesta = 3 * actual + 1;
   else
      respuesta = actual / 2;
   return respuesta;
}
```

4.6 RECURSIVIDAD.

Técnica para resolver un problema que se basa en que **el problema se expresa como una solución inicial y el resto de soluciones se basan en resolver el mismo problema modificado hasta llegar a esa solución inicial**. Ejemplo: La potencia positiva de un número es base elevado a exponente donde exponente es un entero positivo (mayor o igual que cero).

base * base * ... (exponente veces multiplicado ... * base

Así que 2^4 = 2 * 2 * 2 * 2 (2 multiplicado 4 veces con sigo mismo).

En Java:

```
/* precondición: exponente >= 0 */
potencia= 1.0;
for(int i= 0; i < exponente; i++) { potencia *= base; }
```

Eso expresado de forma recursiva:

$$\text{Potencia}(b,e) = \begin{cases} 1, & \text{si } e = 0 \text{ (caso inicial)} \\ b * \text{potencia}(b, e-1) & \text{en otro caso (general)} \end{cases}$$

Es decir: potencia(2,4) = 2 * potencia(2, 3)
= 2 * 2 * potencia(2, 2)
= 2 * 2 * 2 * potencia(2,1)
= 2 * 2 * 2 * 2 * potencia(2,0)
= 2 * 2 * 2 * 2 * 1
= 2 * 2 * 2 * 2

- **PROS**: mecanismo muy potente y elegante.
- **CONTRAS:** menos eficiente que versiones no recursivas del mismo algoritmo.

EJEMPLO 1: Las Torres de Hanoi. Este problema se suele plantear a menudo en programación, especialmente para explicar la recursividad. En su forma más tradicional, consiste en 3 postes verticales. En uno de los postes se apila un número indeterminado de discos perforados por su centro (elaborados de madera), que determinará la complejidad de la solución. Por regla general se consideran siete discos. Los discos se apilan sobre uno de los postes en tamaño decreciente de abajo a arriba. No hay dos discos iguales, y todos ellos están apilados de mayor a menor radio -desde la base del poste hacia arriba- en uno de los postes (A), quedando los otros dos postes vacíos (B y C). El juego consiste en pasar todos los discos desde el poste ocupado (es decir, el que posee la torre) a uno de los otros postes vacíos. Para realizar este objetivo, es necesario seguir tres simples reglas:

- Solo se puede mover un disco cada vez.
- Un disco de mayor tamaño no puede estar sobre uno más pequeño que él mismo.
- Solo se puede desplazar el disco que se encuentre arriba en cada poste.

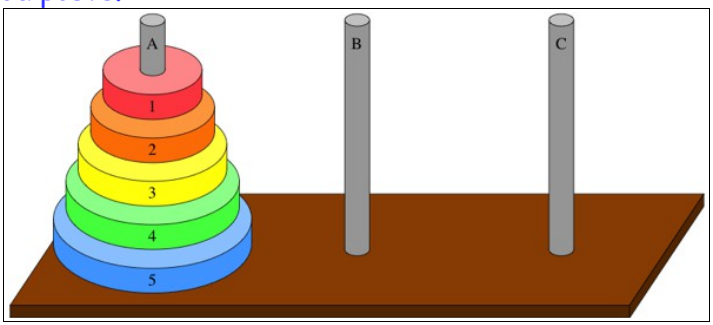


Figura 2: Juego de las Torres de Hanoi.

Aunque el origen del juego es de un matemático francés, se suele adornar con leyendas como:

"En el gran templo de Benarés, debajo de la cúpula que marca el centro del mundo, yace una base de bronce, en donde se encuentran acomodadas tres agujas de diamante, cada una del grueso del cuerpo de una abeja y de una altura de 50 cm aproximadamente. En una de estas agujas, Dios, en el momento de la creación, colocó sesenta y cuatro discos de oro -el mayor sobre la base de bronce, y el resto de menor tamaño conforme se va ascendiendo-. Día y noche, incesantemente, los sacerdotes del templo se turnan en el trabajo de mover los discos de una aguja a otra de acuerdo con las leyes impuestas e inmutables de Brahma, que requieren que siempre haya algún sacerdote trabajando, que no muevan más de un disco a la vez y que deben colocar cada disco en alguna de las agujas de modo que no cubra a un disco de radio menor. Cuando los sesenta y cuatro discos hayan sido transferidos de la aguja en la que Dios los colocó en el momento de la creación, a otra aguja, el templo y los brahmanes se convertirán en polvo y junto con ellos, el mundo desaparecerá...

Existen diversas formas de llegar a la solución final, todas ellas siguiendo estrategias diferentes.

UNA SOLUCIÓN RECURSIVA.

Si numeramos los discos desde 1 hasta *n*, si llamamos *origen* a la primera pila de discos (izquierda), *destino* a la tercera (derecha) y *auxiliar* a la intermedia (centro), y si a la función la denomináramos *hanoi()*, con *origen*, *auxiliar* y *destino* como parámetros, el algoritmo de la función sería el siguiente:

Algoritmo Torres de Hanói
Entrada: Tres pilas de números llamadas *origen*, *auxiliar*, *destino* (pila *origen* ordenada)
Salida: La pila *destino*

SI *origen == 1*

ENTONCES
mover el disco 1 de pila origen a la pila destino (insertarlo arriba de la pila destino)
SI NO
// mover todas las fichas menos la más grande (*n*) a la varilla
// auxiliar
hanoi(*origen*, *destino*, *auxiliar*)
// mover la ficha grande hasta la varilla final
mover disco *n* a *destino*
// mover todas las fichas restantes, 1...n–1, encima de la
// ficha grande (n)
hanoi(*auxiliar*, *origen*, *destino*)
FIN SI

El número de movimientos mínimo a realizar para resolver el problema de este modo es de 2^{n-1}, siendo *n* el número de discos. El programa en Java:

```
package TorresHanoi;

import javax.swing.JOptionPane;

public class TorresDeHanoi {
   private int NumeroDeDiscos;       // Nº de discos
   private int NumeroDeMovimientos; // Nº movimientos realizados

   public int getNumeroDeDiscos(){ return NumeroDeDiscos; }

   public void setNumeroDeDiscos(int NumeroDeDiscos) {
      this.NumeroDeDiscos = NumeroDeDiscos;
    }

   public int getNumeroDeMovimientos() {
      return NumeroDeMovimientos;
   }

   public void setNumeroDeMovimientos(int NumeroDeMovimientos) {
      this.NumeroDeMovimientos = NumeroDeMovimientos;
```

```
}

public void Captura(){
   NumeroDeDiscos = Integer.parseInt(
                JOptionPane.showInputDialog("¿CUANTOS DISCOS?")
                     );
}

public void Intercambio(int NumDiscos, char A, char B, char C){
   // Los parámetros se envían a la segunda clase
   /* Tomemos en cuenta que el parámetro A, tomará el lugar de
      la torre inicio, la que inicialmente contiene todos los
      discos; el parámetro B, será la torre auxiliar; y el
      parámetro C será la torre destino, donde quedarán
      al final de juego todas las fichas */
   if (NumDiscos==1){
      /* si el número de discos es igual a uno, lógicamente se
         moverá el disco de la torre inicio directamente a la
         de destino*/
      setNumeroDeMovimientos( getNumeroDeMovimientos() + 1 );
      JoptionPane.showMessageDialog(null, "Mover disco " +
              NumDiscos + " de la torre " + A + " a la torre "
              + C + "\nMOVIMIENTOS: " + NumeroDeMovimientos);
   }
   else{
      /* se realizarán mas movimientos y serán los siguientes*/
      Intercambio(NumDiscos-1, A, C, B);
      /*... y entonces el método se llama a sí mismo, moviendo
        primero, el disco de la torre A, inicio, a la torre C,
        destino */
      setNumeroDeMovimientos(getNumeroDeMovimientos()+1);
      JoptionPane.showMessageDialog(null, "Mover disco " +
         NumDiscos + " de la torre " + A + " a la torre " + C +
         "\nMOVIMIENTOS: " + NumeroDeMovimientos);
      Intercambio(NumDiscos-1, B, A, C);
      /*... el método vuelve a llamarse a sí mismo, esta vez
        los parámetros donde de se encontraba la variable
        A ahora estará la variable B, para indicar el siguiente
        movimiento*/
   }
}

public void Movimientos() {
```

```
            JoptionPane.showMessageDialog(null, "TOTAL DE MOVIMIENTOS: "
                        + NumeroDeMovimientos);
        }
    }
```

Esta otra clase será la que tenga la subrutina main():

```
package TorresHanoi;

import javax.swing.JOptionPane;

public class Jugar {

    public static void main( String arg[] ) {
        TorresDeHanoi k;
        k= new TorresDeHanoi();
        // se declara y crea la variable de la primera clase
        k.Captura();
        k.Intercambio( k.getNumeroDeDiscos(),'A','B','C');
        // se reciben los parámetros de el método Intercambio
        k.Movimientos();
    }
}
```

SOLUCIÓN NO RECURSIVA

Otra manera de resolver el problema, sin utilizar la recursividad, se basa en el hecho de que para obtener la solución más corta, es necesario mover el disco más pequeño en todos los pasos impares, mientras que en los pasos pares solo existe un movimiento posible que no lo incluye. El problema se reduce a decidir en cada paso impar a cuál de las dos pilas posibles se desplazará el disco pequeño. El algoritmo en cuestión depende del número de discos del problema:

> ***Nota***: *Eliminar la recursividad de un algoritmo que resuelve fácilmente (pero de forma no eficiente) un problema, suele ser una de los ejercicios típicos cuando estudias algoritmos.*

4.7 API'S Y PACKAGES.

Puedes escribir programas con una sencilla interfaz de usuario que utilice la consola. Pero las aplicaciones con GUI tienen ventanas, botones, barras de desplazamiento, menús, cajas de edición de texto y otros muchos elementos que hacen más sencilla la utilización al usuario pero obligan al programador a manejar un abanico mucho más amplio de posibilidades. El programador experimenta un crecimiento importante de la complejidad al aparecer un mayor número de subrutinas para gestionar la interfaz de usuario y otros propósitos.

Toolboxes (Cajas de Herramientas)

Alguien que necesite programas para ordenadores Macintosh—generar programas que se vean y comporten de la forma que el usuario espera—tendrán que usar el Macintosh Toolbox, una colección de cientos de diferentes subrutinas que le permitirán abrir ventanas, dibujar figuras y texto, añadir botones a las ventanas, responder a eventos del ratón, etc. Hay otras rutinas para crear menús y reaccionar a las opciones que elija el usuario. Al margen de la GUI, necesitará rutinas para trabajar con ficheros, comunicarse por la red, enviar información a un la impresora, comunicarse con otros programas, y hacer muchas otras cosas que las aplicaciones necesitan hacer. Microsoft Windows ofrece su propio conjunto de subrutinas a los programadores y son un poco diferentes de las que usan los sitemas Mac. Linux también tienen diferentes Toolboxes para que los usen los programadores.

Cada proyecto necesita una mezcla de innovación y reutilización de herramientas existentes. Un programador que tenga un conjunto de herramientas para usar, comenzará con estas herramientas preconstruidas en el lenguaje como variables, sentencias de asignación, sentencias de control, etc. A esto el programador añade toolboxes con

cientos de subrutinas ya creadas que realizan ciertas tareas. Si estas herramientas están bien diseñadas, pueden utilizarse como cajas negras: puedes solicitarles un trabajo aunque no sepas como lo hacen. La parte de innovación consiste en coger todas estas herramientas y aplicarlas a un proyecto concreto que resuelva un problema (procesar textos, almacenar cuentas y operaciones bancarias, procesar imágenes, navegar por la web, juegos de ordenador, etc.). Esto es programar aplicaciones.

API

Una toolbox de software es un tipo de caja negra que ofrece una interfaz al programador. La interfaz es una especificación de las rutinas que tiene, los parámetros que hay que pasarles y las tareas que realizan. Esta información es la **API (Application Programming Interface)** asociada a la toolbox.

El lenguaje Java ofrece una gran API estándar. Ya hemos utilizado partes de esa API como las subrutinas matemáticas (Math.sqrt() por ejemplo) y la salida de datos por consola (System.out.print()). El API estándar de Java incluye rutinas para trabajar con GUI, comunicaciones de red, lectura y escritura de ficheros, etc.

Como Java es independiente de la plataforma (hardware y sistema operativo) el mismo programa debe ejecutarse en Mac OS, Windows, Linux y otros entornos. La misma API debe funcionar en todas las plataformas. Pero lo que realmente es independiente de la plataforma es la interfaz, no la implementación. Si te aprendes y usas la API, tendrás programas que funcionen en muchas plataformas.

PACKAGE

Las subrutinas de la API de Java se agrupan en clases. Para ofrecer

una organización a gran escala, las clases se pueden agrupar en paquetes. **Un paquete también puede contener a otros paquetes.** Toda la API estándar de Java está implementada en varios paquetes. Uno de ellos se llama **java** y contiene muchos paquetes que no son para GUI y el paquete original para GUI que se llama "AWT". Otro package, "javax", se ha añadido a la versión 1.2 y contiene las clases usadas por Swing, una toolbox para GUI.

Un package puede contener clases y otros paquetes. El paquete contenido se llama un "sub-package." Tanto el paquete **java** como **javax** contienen subpaquetes. Uno de los subpaquetes de java es "awt". Su ruta completa es: **java.awt**. Aunque ya no se utiliza, AWT contiene algunas clases como **Graphics** que contiene las rutinas para dibujar en la pantalla, **Color** para definir colores y **Font** para trabajar con texto de forma gráfica. Sus rutas completas serán: **java.awt.Graphics**, **java.awt.Color** y **java.awt.Font**.

De igual forma, **javax** contiene un subpaquete llamado **javax.swing**, que incluye clases para la GUI como **javax.swing.JButton**, **javax.swing.JMenu** y **javax.swing.JFrame**. Las clases de javax.swing, junto con las clases básicas de java.awt, son toda la parte de la API de Java que hace posible escribir programas GUI en Java.

El paquete **java** contiene otros subpaquetes como **java.io** (para entrada/salida de datos), **java.net** (comunicaciones de red) y **java.util** (una variedad de clases de utilidad). Otro subpaquete básico está en **java.lang** (contiene clases como String, Math, Integer y Double). Si miras la organización de forma gráfica, quizás sea más sencilla.

Por ejemplo, la documentación de la API de Java 8 tiene más de 210 paquetes con más de 4025 clases.

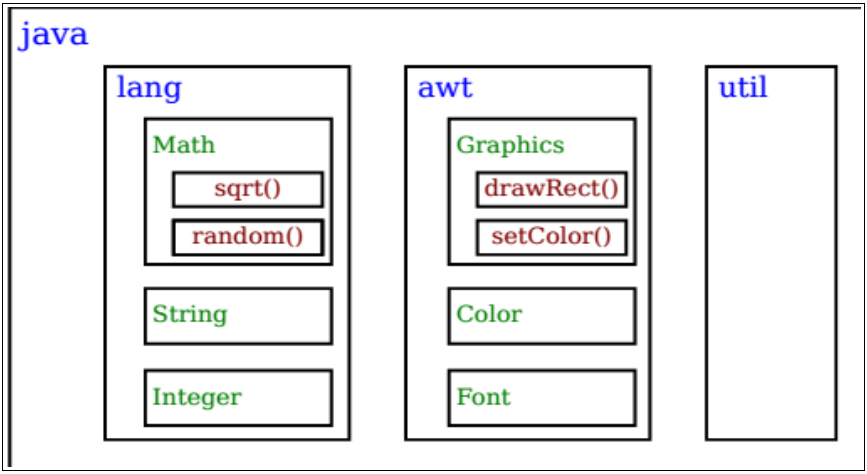


Figura 3: Esquema de algunos paquetes de la API estándar de Java.

Muchas de ellas son oscuras o muy especializadas, nadie las domina todas, como mucho las investigas si alguna vez las necesitas. Si quieres ojearlas puedes usar este enlace:

http://download.oracle.com/javase/8/docs/api/

Si cambias el 8 de la URL por otra versión probablemente vayas a la documentación de la API de esa versión de java.

USANDO CLASES DE LOS PAQUETES (directiva import)

Imagina que quieres usar la clase **java.awt.Color** en un programa. Como cualquier clase, **java.awt.Color** es como un tipo (puedes usarlo para declarar variables y parámetros). Una forma de hacerlo es usar la ruta completa a la clase. Por ejemplo si quieres declarar una variable color1:

```
java.awt.Color color1;
```

Para no tener que usar las rutas completas a las clases puedes importar las clases de un paquete al principio del fichero de código

fuente y posteriormente solamente debes poner el nombre de la clase, sin la ruta. Por ejemplo:

```
import java.awt.Color;
...
Color color1;
```

***Nota**: las sentencias import aparecen al principio del fichero de código fuente (como mucho detrás de la sentencia **package** si la hay) y fuera de cualquier clase.*

Más que una sentencia, import es una directiva. Se puede usar una abreviatura para importar todas las clases bajo un paquete, que es el carácter asterisco y significa todas las clases del paquete.

```
import java.awt.*;
```

Nota: no importa las clases de subpaquetes.

Nota: algunos programadores piensan que usar el asterisco es una mala opción porque importas clases que no usas, que es mejor importar solo aquellas clases que usas de forma individual. Que no te sepa mal usarlo, es lo habitual.

Por ejemplo es muy común ver programas GUI que hagan esto:

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
```

Si debes ser cuidadoso al usar el asterisco al importar en estos casos:

- Que dos clases se llamen igual en diferentes paquetes importados con asterisco. Por ejemplo: java.awt y java.util contienen una clase llamada **List**. Si importas java.awt.* y java.util.*, al usar solamente el nombre List, se produce una ambiguedad. En este caso, debes:

- Usar el nombre completo de la clase (java.awt.List o java.util.List).
- No importar con asterisco si solo necesitas una de las dos.

Como las clases del paquete **java.lang** son fundamentales, se importan automáticamente. Así que es como si cada programa Java comenzase con la sentencia:

import java.lang.*;

AGRUPAR CLASES EN PAQUETES (directiva package)

Un programador puede crear nuevos paquetes. Si tu quieres que algunas de las clases que estás escribiendo estén dentro de un paquete llamado utilidades, entonces, tus ficheros de código fuente deben comenzar con la sentencia:

```
package nombre_del_paquete;
```

Debe aparecer antes que cualquier directiva import en el fichero y el código fuente del fichero debe colocarse en una carpeta con el mismo nombre que el paquete y las clases que estén en un subpaquete deben estar en una subcarpeta. Por ejemplo: una clase que esté en un paquete llamado utilidades.net debe estar en una carpeta llamada net dento de una carpeta llamada utilidades.

Las clases de un paquete tienen acceso automáticamente a otras clases del mismo paquete, es decir, no tienes que importarlo.

En las versiones recientes de Java, los paquetes se almacenan en archivos jar en una subcarpeta del Java Runtime Environment, como rt.jar.

Si una clase no indica de forma explícita un package, se ubica en el

denominado paquete por defecto, que no tiene nombre.

Cuando tenemos un programa grande (con muchas clases), lo ideal es meter todos estos ficheros y directorios en un único fichero comprimido. La herramienta **jar** del JDK nos permite hacer esto. Empaqueta todo lo que le digamos (directorios, clases, ficheros de imagen o lo que queramos) en un único fichero de extensión **.jar**. Lo bueno es que puedes ejecutar el fichero directamente sin tener que descomprimirlo.

EJEMPLO: Crear un ejecutable .jar desde línea de comandos. Creamos un proyecto con 2 clases C1 (la que se ejecuta) y C2 (usada por C1) en la carpeta de nombre "pami" (**pa**ckage **mi**o).

```
// Fichero C1.java
package pami;

public class C1 {
    public static void main(String[] args) {
        C2 c2 = new C2();
        System.out.println( c2.saluda() );
    }
}

// Fichero C2.java
package pami;

public class C2 {
    public String saluda() { return "Hola, soy C2";   }
}
```

Entramos en la carpeta pami y compilamos ambas clases para generar los ficheros .class:

```
cd .\pami
javac C1.java C2.java
```

Ahora debemos crear un fichero manifest. Es un fichero de texto con extensión .mf que describe caracteríscas de la aplicación que

empaqueta el .jar, por ejemplo, qué clase es la que debe ejecutarse. Creamos el fichero **tmp.mf** con este contenido en la carpeta pami:

```
Main-Class: pami.C1
```

La primera línea indica que la clase ejecutable es C1 del paquete pami (observa que no tiene .java).

Nos vamos a la carpeta padre y creamos el jar y añadimos todos los ficheros (si tu programa usa imágenes, sonidos, vídeos...) y subcarpetas de la carpeta pami indicando el nombre de la carpeta:

```
jar -cf pami.jar pami
```

O solo los de las clases y no el resto de ficheros:

```
jar -cf pami.jar pami\*.class
```

También podemos modificar cosas, por ejemplo si tenemos que recompilar C2, con la opción u lo actualizamos (update):

```
jar -uf pami.jar pami\C2.class
```

Ya puedes ejecutarlo, pero debes indicar el nombre de la clase. Para ahorrarnos este trabajo hemos creado el manifest. Para añadirlo usamos el comando:

```
jar -umf pami\tmp.mf
```

También podríamos haberlo incluido al crear el jar:

```
jar -cmf pami/tmp.mf pami.jar pami\*.class
```

Por último para ejecutarlo:

```
java -jar pami.jar
```

4.8. OBJETOS.

Mientras que una subrutina representa una tarea sencilla, un objeto encapsula tanto datos (variables de instancia) y varias tareas (comportamiento, en forma de métodos de instancia). Así que los objetos son otra estructura que se usa para ayudar a manejar la complejidad de programas grandes.

Hay varias formas de crear objetos, una es la que usan la mayoría de los lenguajes orientados a objetos, **los objetos se crean cuando se instancia una clase**. La clase es por tanto una forma de describir los objetos que se pueden instanciar, como una plantilla de sus objetos. Aunque también tienen otros usos. Otros lenguajes (como javascript) utilizan la técnica de prototipado, que es una especie de herencia.

4.8.1. MÉTODOS Y VARIABLES DE INSTANCIA.

Si un objeto es un grupo de datos y de subrutinas, ¿Qué diferencia hay entre una clase y un objeto?

Las clases describen a los objetos, o de forma más exacta, sus partes no estáticas describen a los objetos. La terminología más común dice que los objetos pertenecen a una clase. Pero desde el punto de vista de un programador, **las clases se usan para crear los objetos**. Una clase es el diseño de un objeto, cuyas partes no estáticas indican o describen qué variables y métodos contendrán los objetos. Así que la principal diferencia entre clases y objetos es que los objetos se crean y se destruyen durante la ejecución de un programa y muchos objetos creados a partir de la misma clase comparten la misma estructura.

Considera una sencilla clase que describe al usuario que está utilizando un programa:

```
class Usuario {
```

```
    static String nombre;
    static int edad;
}
```

Si un programa usa esta clase, solo hay una copia de las variables Usuario.nombre y Usuario.edad. Tendrá una zona de memoria dedicada a almacenar los valores de estas variables. Podemos dibujar la clase en la memoria:

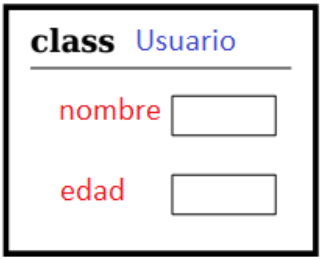


Figura 1: dibujo de una clase sencilla en memoria.

Si el programa quisiera manejar los datos de varios usuarios, con la clase diseñada así no podría, solo sirve para representar a un usuario. Podrías pensar en cambiar cada dato por un array y así tener varios. Pero el enfoque de la programación orientada a objetos es que cada cosa del mundo real, se corresponda con un objeto. Además, la clase existe desde que el programa comienza a funcionar hasta que acaba de ejecutarse. Lo idel sería que un usuario existiera desde que se necesita trabajar con él, hasta que ya no se necesita. Ahora, vamos rediseñar la clase para que tenga variables miembro no estáticas:

```
class Jugador {
    static int numJugadores;
    String nombre;
    int edad;
}
```

Ahora esta clase solamente tiene una variable estática (de la clase).

Los datos nombre y edad ya no son de la clase (por eso no poderemos referenciarlos como Jugador.nombre y Jugador.edad). Estas variables no estáticas son de cada objeto que se cree a partir de la clase, cada uno debe tener su propia memoria para almacenar los valores. Así que la parte no estática de una clase es una plantilla de sus objetos:

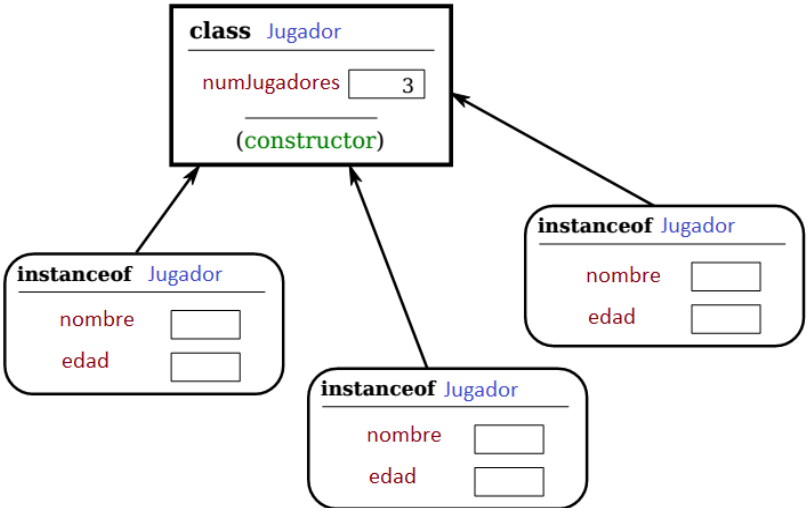


Figura 2: dibujo de una clase y 3 objetos suyos en memoria.

Un objeto creado a partir de una clase se denomina **una instancia de la clase** que significa una existencia real de la plantilla (se dibuja con una flecha que sale del objeto y apunta a su clase). En la figura, la clase contiene algo denominado **constructor** que indica la exisencia de una rutina en la clase encargada de crear los objetos (constructores).

Las variables que son de cada objeto se llaman **variables de instancia** y los métodos que son de los objetos se llaman **métodos de instancia**. Aunque el ejemplo no incluye ningún método, la mecánica es idéntica: si tiene el modificador **static** es de la clase, si no, es de los objetos.

En la vida real se dan todos los casos: verás clases que solo tienen miembros estáticos, mezclados o solamente miembros de instancia.

4.8.2 DIFERENCIAS ENTRE CLASES Y OJETOS.

Vamos a comenzar con una versión simplificada de una clase que represente a un estudiantes de un curso:

```
public class Alumno {
   public String nombre;  // Nombre del alumno
   public double eval1, eval2, eval3; // Notas
   public double getMedia() { // calcula la nota media
      return (eval1 + eval2 + eval3) / 3;
   }
} // fin Alumno
```

Ninguno de los miembros es estático, así que esta clase solo sirve para crear objetos. En Java, una clase es un tipo de dato. Así que podemos usarlo en la sentencia de declaración de variables, o como tipo de un parámetro formal en una subrutina, o el dato devuelto en una función con la sentencia **return**.

```
Alumno a1;
```

Sin embargo, en el caso de los objetos, declarar una variable no crea el objeto: en Java, una variable no almacena un objeto, solamente lo referencia (señala o apunta a la zona de memoria donde está almacenado). Los objetos se almacenan en una zona de memoria llamada **heap**. La variable tiene información para localizarlo, no lo almacena.

En un programa los objetos se crean usando el operador **new**, que crea el objeto y devuelve una referencia a la memoria donde existe. El operador **new** llama a una subrutina especial llamada **constructor**. Ej:

```
a1 = new Alumno();
```

Como la variable a1 referencia a un objeto o instancia de la clase

Alumno, para acceder a sus datos y métodos debemos usar la variable que los referencia y el nombre de las variables y métodos, por ejemplo:

```
System.out.println("Hola, " + a1.nombre + ". Tus notas son:");
System.out.println(a1.eval1);
System.out.println(a1.eval2);
System.out.println(a1.eval3);
```

Lo mismo se aplica a los métodos:

```
System.out.println( "Tu media es " + a1.getMedia() );
```

Puedes usar los datos en expresiones según el tipo que tengan. Ej:

```
a1.nombre.length()
```

Cuando una variable de tipo clase, no referencia a ningún objeto, decimos que es una referencia nula, o un puntero a valor **null**. Puedes almacenar null en una variable de tipo referencia:

```
a1 = null;
```

Para saber si una variable está referenciando a un objeto, puedes comprobarlo. Ej:

```
if ( a1 == null ) ...
```

Nota: *si intentas acceder a un miembro de un objeto a través de una referencia nula, se lanza una excepción de puntero nulo* ***NullPointerException***.

Vamos a ver una secuencia de sentencias que trabajan con objetos:

```
Alumno a, a1, a2, a3; // Declara 4 variables de tipo Alumno
a = new Alumno();     // Crea un objeto y guarda la referencia
a1 = new Alumno();    // Crea un segundo objeto Alumno
a2 = a1;              // Copia la referencia en a2
a3 = null;            // a3 no referencia nada
a.nombre = "John Smith"; // guarda un nombre en el objeto a1
a1.nombre = "Mary Jones";
```

```
// El resto de variables de instancia tiene valor por defecto 0.0
```

Lo que estas sentencias consiguen, representado en memoria sería:

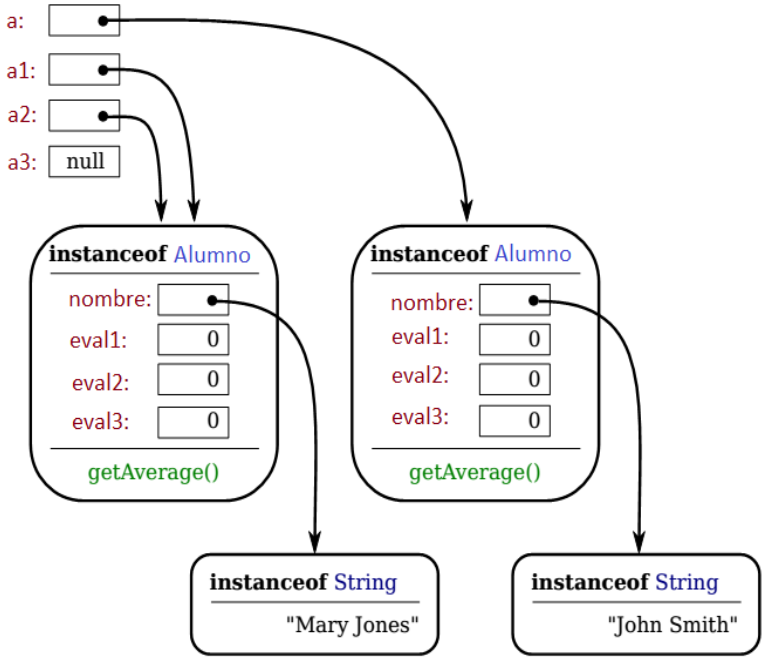


Figura 3: aspecto en memoria de variables referenciando objetos.

Las referencias se han representado mediante flechas. Observa que los String son objetos, por tanto las variables también son referencias. Pero lo interesante de esta figura es lo que ocurre en a1 y en a2: **cuando una variable objeto se asigna a otra, se copia la referencia, no el objeto**. Las dos variables acaban apuntando al mismo objeto, no se ha creado un nuevo objeto clonando al original, es el mismo. Por tanto si cambias algo del objeto de a1, estás cambiando también el objeto de a2, porque en realidad son el mismo.

Puedes comprobar la igualdad o desigualdad de dos objetos usando el

operador == y !=, pero de nuevo debes comprender bien lo que haces, lo que comparas son las referencias, no los objetos, comparas si estás refenciando al mismo objeto, no estás comprobando si los objetos que referencias tienen los mismos valores. Para comprobar si los objetos a1 y a2 tienen los mismos valores debes hacer:

```
a1.eval1 == a2.eval1 && a1.eval2 == a2.eval2 &&
a1.eval3 == a2.eval3 && a1.nombre.equals(a2.nombre)
```

Observa que aunque los string son objetos, aunque lo único que almacenan es la cadena de caracteres, si usas los comparadores de igualdad y desigualdad puedes tener problemas, porque estás comparando no las cadenas de caracteres si no las referencias a esas cadenas. Ej:

```
String s1 = "Hola";
String s2 = "Hola";
if( s1 == s2 )  {
   System.out.println("Son iguales");
} else {
   System.out.println("No son iguales");  // el resultado!!
}
```

Imagina que una variable que referencia a un objeto se declara como **final**. Eso significa que el valor almacenado nunca puede ser cambiado tras inicializar la variable. Esto significa que siempre referencia al mismo objeto, pero no impide que el objeto cambie. Ej:

```
final Alumno a = new Alumno();
a.nombre = "John Doe"; // Cambia el objeto, no la referencia
```

Ahora, imaginemos que la variable **obj** la pasamos a una subrutina (obj referencia a un objeto), ya sabemos que la subrutina no puede cambiar el valor de la variable, puesto que trabaja con una copia. Pero, la copia es de la referencia, no del objeto. Luego si cambias el objeto, dentro de la subrutina, los cambios son visibles en el exterior. Ej:

```
void noCambia(int z) {
      z = 42;
}
```

```
void cambia(Alumno a) {
    a.nombre = "Fred";
 }
```

Las llamadas:

```
x = 17;
a.nombre = "Jane";
noCambia(x);
cambia(a);
System.out.println(x);
System.out.println(a.nombre);
```

Salida: 17 y "Fred".

La palabra reservada **this** representa al objeto actual que está ejecutando una sentencia. Por tanto, no se puede usar desde dentro de un bloque de sentencias estático (es de la clase y this no tiene sentido).

4.8.3. GETTERS Y SETTERS

Cuando se crea una nueva clase hay que prestar atención al acceso que se permite a las variables. Java permite tres posibilidades de control de acceso: ***public*** significa que se tiene acceso desde cualquier lugar; ***private*** restringe el acceso a las subrutinas de la clase; no indicar nada permite acceso desde cualquier clase del mismo paquete.

La encapsulación (agrupar cosas) y la ocultación de detalles son principios de la orientación a objetos. La mayoría de las variables de una clase deberían ser privadas. Y para permitir que desde fuera se puedan leer/modificar, la propia clase aporta métodos que hacen posible estas operaciones. Por ejemplo, si una clase tiene una variable privada de tipo String y de nombre **titulo**, puedes crear este método para permitir leerla:

```
public String getTitulo() {
   return titulo;
}
```

A estos métodos se les llama **getters** y su nombre suele construirse con la palabra "get" seguida del nombre de la variable capitalizado, en el ejemplo "get" + "Titulo" -> getTitulo().

Si quieres permitir la operación de escritura (cambiar el valor), puedes implementar un método que lo haga. A estos métodos se les llama **setters** y su nombre se construye con la palabra "set" y el nombre de la variable capitalizado. En el ejemplo:

```
public void setTitulo( String nuevoTitulo ) {
   titulo = nuevoTitulo;
}
```

A veces, a las variables de tipo boolean, se les suele proporcionar getters que no comienzan con "get" si no con "is". Por ejemplo una variable privada booleana que se llame cobrada. Su getter podría ser: isCobrada() (está cobrada).

La ventaja de implementar getters y setters en vez de declarar las variables públicas es que en las subrutinas, además de permitir leer/escribir los datos, también puedes hacer más cosas, por ejemplo contar cuantas veces ha sido modificada una variable, o la fecha y hora, o quien accede. Por ejemplo:

```
public String getTitulo() {
   leenTitulo++;
   return titulo;
}
```

y un setter puede hacer verificaciones sobre la legalidad del valor a cambiar:

```
public void setTitulo( String nuevoTitulo ) {
```

```
        if ( nuevoTitulo == null )
           titulo = "(Sin título)"; // un valor por defecto
        else
           titulo = nuevoTitulo;
     }
```

Incluso **pueden ayudarte a no tener que cambiar código si en tus clases los usas desde el principio**. Imagina que haces una clase con dos variables, usas la clase durante años en tus programas, que acceden a sus variables no directamente si no a través de los getters y setters **(los programas no acceden, no conocen, no dependen del interior de la clase, has ocultado su implementación)**. Al cabo de dos años decides (por un buen motivo) cambiar las variables que contiene. ¿Eso implica que debes modificar todos los programas que la han usado durante esos dos años? No, simplemente debes mantener los getters y setters que usaban. En ese caso, **los cambios se aislan de forma eficiente**.

Los getters y setters **definen una propiedad de la clase** que podría corresponderse internamente con una, varias o ninguna variable en realidad. Eso queda oculto al exterior (encapsular y ocultar).

4.8.4 ARRAYS Y STRINGS SON OBJETOS.

Ya hemos trabajado con Strings (los operadores de igualdad comprueban si apuntan al mismo objeto, no si su lista de caracteres coincide, etc.) y arrays, que también son objetos y se les aplica las mismas propiedades que a cualquier objeto, así que:

- Una variable array es solo una referencia a la memoria del heap donde se almacena el objeto con sus valores.
- Nunca es posible pasar un array por valor cuando lo usas como parámetro actual.

- Si asignas un array, no copias los elementos, simplemente se copia la referencia (las variables apuntan al mismo objeto).
- Si accedes a un elemento o a la propiedad **length** de un array que no referencia a nada, se genera la excepción **NullPointerException**.
- Para saber si un array no referencia, puedes comparar con null.
- Si comparas dos arrays con el operador de igualdad, no estás comparando sus elementos, sino si son el mismo objeto.

Ejemplos:

```
int[] lista = { 1, 2, 3};
int[] nuevaLista;
nuevaLista = lista;  // Las dos apuntan al mismo objeto
nuevaLista[1] = 0;
System.out.println( lista[1] ); // Imprime 0
```

Los arrays son objetos que pueden almacenar objetos, es decir su tipo base puede ser una clase. Podemos tener arrays de cadenas String[] o de cualquier otra clase. Podemos tener por ejemplo un array de la clase Alumno que definimos antes: Alumno[]

```
Alumno[] aula;          // Declara una variable array de alumnos
aula = new Alumno[30]; // ahora referencia un objeto array
```

Cuando se crea el array de objetos, los elementos se inicializan con el valor por defecto de los objetos que es null, así que aula[0] ... aula[29] tienen null. Para rellenar los elementos, habrá que crear 30 objetos de la clase Alumno (instanciar la clase 30 veces):

```
Alumno[] aula;
aula = new Alumno[30];
for ( int i = 0; i < 30; i++ ) {
   aula[i] = new Alumno();
}
```

Para acceder al alumno de la posición 3 podemos usar aula[3] que es a todos los efectos un objeto de la clase Alumno:

```
aula[3].nombre; // nombre del Alumno de la posición 3 del array
```

4.9. CONSTRUCTORES E INICIALIZACIÓN.

Los objetos, al contrario que los tipos de datos primitivos, además de declararse, para existir deben crearse de forma explícita. La creación implica buscar memoria en el heap para almacenarlo, y dar un primer valor a sus variables de instancia. La memoria donde se quede almacenado, en principio te da igual, pero si te interesa en muchas ocasiones controlar qué valores se les da a las variables.

4.9.1 INICIALIZAR VARIABLES.

A una variable de instancia se le puede asignar un valor inicial al declararla como a cualquier otra variable. Igual ocurre con las variables estáticas, que se pueden inicializar al cargar la clase. Como salvo declaraciones e inicializaciones no puede haber sentencias que no estén en un bloque, podríamos tener esto:

```
public class Lanza2Dados {
   static public numLanzamientos = 0; // Al cargar la clase
   public int dado1 = 3; // Nº del primer dado, al crear objeto
   public int dado2;     // Nº del segundo dado
   static { numLanzamientos++; } // Al cargar la clase
   { dado2 = 4; }                // Al crear cada objeto
}
```

Ahora hacemos una nueva versión de Lanza2Dados que representa el lanzamiento de dos dados. Tiene dos variables de instancia que contienen los números actuales de los dados (**dado1** y **dado2**) y un método **lanza()** que simula su lanzamiento:

```
public class Lanza2Dados {
```

```
    public int dado1 = 3; // Nº del primer dado
    public int dado2 = 4; // Nº del segundo dado
    public void lanza() {
       dado1 = (int)(Math.random() * 6) + 1;
       dado2 = (int)(Math.random() * 6) + 1;
    }
} // fin clase
```

La inicialización de las variables de instancia se ejecuta cuando el objeto es creado. Una variación de la clase:

```
public class Lanza2Dados {
    public int dado1 = (int)(Math.random() * 6) + 1;
    public int dado2 = (int)(Math.random() * 6) + 1;
    public void lanza() {
       dado1 = (int)(Math.random()*6) + 1;
       dado2 = (int)(Math.random()*6) + 1;
    }
} // fin clase
```

Ahora cada nuevo objeto probablemente tenga valores diferentes a otro, al asignarlos de forma aleatoria. Si no das un valor inicial a las variables, se les da el valor por defecto del tipo que tengan (0 para tipos numéricos, el carácter UNICODE 0 para char, false para booleanos y null para objetos).

4.9.2 CONSTRUCTORES.

Ya sabes que los objetos se crean con el operador **new**. Por ejemplo:

```
Lanza2dados dado;          // Declara una variable
dado = new Lanza2Dados(); // Construye objeto y deja referencia
```

En este ejemplo, la expresión **new Lanza2dados()** le busca memoria al objeto, inicializa sus variables de instancia y devuelve la referencia al nuevo objeto. La referencia se guarda en la variable dado. El método **Lanza2Dados()** se dice que es un **constructor**. Existe aunque nadie la ha definido, es decir, **toda clase tiene como mínimo un constructor**, **si el programador no lo define, se crea uno de forma automática**

(**el constructor por defecto**). El constructor por defecto solo hace cosas básicas (aloja memoria e inicializa las variables). Una clase puede definir más de un método constructor.

Un constructor es como cualquier otro método salvo por 3 cosas:

1. No devuelve ningún dato, ni siquiera **void**.
2. El nombre siempre coincide con el de la clase.
3. Solo puede tener modificadores de acceso (**public**, **private** y **protected**).

Todo lo demás lo comparte con los métodos: tiene un bloque de sentencias y parámetros formales. Los parámetros se usan para pasar información que se utiliza para construir el objeto. Por ejemplo podemos crear uno para que defina el valor inicial de los dados:

```
public class Lanza2dados {
   public int dado1; // Nº del primer dado
   public int dado2; // Nº del segundo dado
   // Constructor: Crea el objeto y da valores de parámetros
   public Lanza2Dados(int valor1, int valor2) {
      dado1 = valor1;
      dado2 = valor2;
   }
   public void lanza() {
      dado1 = (int)(Math.random() * 6) + 1;
      dado2 = (int)(Math.random() * 6) + 1;
   }
} // fin clase
```

El constructor que hemos creado tiene 2 parámetros así que ahora podemos crear objetos de esta forma:

```
Lanza2Dados dados;            // Declarar variable
dados = new Lanza2Dados(1,1); // se crea con valores 1,1
```

Pero ahora que hemos definido un constructor, el constructor por defecto lo hemos perdido (**solo tenemos constructor por defecto si**

no hay definido ninguno), así que si queremos tener uno sin parámetros hay que definirlo. Añadimos a la clase este otro:

```
public Lanza2Dados() {
  lanza(); // llama al método para generar valores
}
```

Las clases una vez escritas y probadas pueden utilizarse en cualquier programa para crear objetos:

```
public class Lanzamientos {

   public static void main(String[] args) {
      Lanza2Dados primero;          // Primer par de dados
      primero = new Lanza2dados();
      Lanza2dados segundo;          // Segundo juego de dados
      segundo = new Lanza2Dados();
      int tiradas;                  // Cuenta los lanzamientos
      int total1;        // Total del primer juego
      int total2;        // Total del segundo juego
      tiradas = 0;
      do {               // lanzar hasta que los totales coincidan
         primero.lanza();
         total1 = primero.dado1 + primero.dado2;
         System.out.println( "Primero sale " + total1 );
         segundo.lanza();
         total2 = segundo.dado1 +segundo.dado2;
         System.out.println("Segundo sale " + total2 );
         lanzamientos++;         // Contar este lanzamiento
         System.out.println();
      } while (total1 != total2);
      System.out.println("Coinciden en " + lanzamientos
                          + " lanzamientos.");
   } // fin main()
} // fin clase
```

Los constructores son subrutinas especiales, no son métodos de instancia (de los objetos) porque no hay objeto cuando se utilizan por primera vez (crearlos es su trabajo). Son más parecidos a los métodos de clase, pero no pueden ser estáticos, así que no son de

la clase. Al final, no se les llama métodos. Además, solo se pueden llamar junto con el operador **new** siguiendo la sintaxis:

new Clase(lista_parámetros);

La llamada a un constructor es más complicada que a un método normal. Veamos los pasos que realiza:

1. Obtiene un bloque de memoria sin usar del heap, lo suficientemente grande para almacenar al objeto.
2. Inicializa las variables de instancia, si la declaración indica un valor se usa, si no, el valor por defecto.
3. Los parámetros actuales del constructor (si los hay) se evalúan y asignan a los parámetros formales.
4. Se ejecutan las sentencias del cuerpo del constructor (si las hay).
5. Se devuelve una referencia al objeto creado.

IMPORTANTE: Si desde un constructor queremos llamar a otro constructor para construir el objeto:

- No sirve de nada usar new porque no lo podremos devolver.
- Dbe ser la primera sentencia que aparezca.
- No podemos usar el nombre de la clase sino **this(...)**

EJEMPLO 1: Redefine clase Alumno y comenta varios trucos/mejoras con el uso de constructores y modificadores de acceso.

```
public class Alumno {
   private String nombre;              // Nombre del alumno
   public double eval1, eval2, eval3; // Notas en evaluaciones
   // Constructor con nombre -> Impide alumnos con nombre a null
   Alumno(String suNombre) {
      if ( suNombre == null )
         throw new IllegalArgumentException("Debo tener nombre");
```

```
            nombre = suNombre;
        }
        public String getNombre() {  // Getter del nombre
            return nombre;
        }
        public double getAverage() {
            return (eval1 + eval2 + eval3) / 3;
        }
    } // fin de clase
```

Primera mejora: No deja que se creen alumnos sin nombre, no hay otra forma de crearlos (obligamos al programador a dar un valor de una propiedad) An object of type Student contains information about some particular student. El constructor en esta clase tiene un parámetro de tipo String, que indica el nombre de estudiante.

Segunda mejora: al usar el modificador private, impedimos que se pueda cambiar el nombre de un alumno y ni siquiera consultarlo. Para dejar leerlo, definimos un getter, pero no dejamos que se cambie. Una vez asignado.

> **Nota**: *es un buen estilo de programación declarar un dato que no queremos que cambie como* ***final****. El único lugar donde está permitido darle un valor es dentro de un constructor, porque se considera el primer valor.*

EJEMPLO 2: Sacar partido de tener datos estáticos (de clase) y de instancia (de los objetos). Necesitamos que cada nuevo alumno tenga un código que lo diferencie del resto de alumnos que haya, es decir, que no se repita en dos alumnos diferentes (como un NIA). ¿Cómo programar eso? Vamos a conseguirlo teniendo ese código NIA como un dato de la clase, cada nuevo objeto que se cree, incrementamos el NIA siguiente a usar.

```
    public class Alumno {
```

```
    private String nombre;              // Nombre del alumno
    public double eval1, eval2, eval3; // Notas en evaluaciones
    private int NIA;                    // Único para cada alumno
    private static int siguienteNIA = 0;
    // Constructor con nombre -> Impide alumnos con nombre a null
    Alumno(String suNombre) {
       if ( suNombre == null )
          throw new IllegalArgumentException("Debo tener nombre");
       nombre = suNombre;
       siguienteNIA++;
       ID = siguienteNIA;
    }
    public String getNombre() {  // Getter del nombre
       return nombre;
    }
    public int getNIA() {        // Getter del NIA
       return NIA;
    }
    public double getAverage() {
       return (eval1 + eval2 + eval3) / 3;
    }
} // fin de clase
```

Nota: *en programas que usen más de un thread (varios hilos y cada hilo creando en paralelo alumnos) no habría garantías de obtener un NIA único para todos, habría que crear zonas de exclusión mutua en el código, pero eso sale del presupuesto...*

4.9.3 RECOLECCIÓN DE BASURA.

Un objeto exise en el heap y puede ser accedido a través de las variables que almacenan su referencia. ¿Qué hacer con un objeto del que no se guarda su referencia? Considera estas sentencias:

```
Alumno a1 = new Alumno("Juanita Pérez");
a1 = null;
```

La segunda sentencia hace que el programa ya no pueda trabajar con el objeto que crea la primera sentencia. La memoria que consume el objeto ya no es útil y si se libera, otros objetos la pueden aprovechar.

Pero Java (al contrario que otros lenguajes como C++) no tiene sentencias para devolver la memoria que ocupan los objetos.

Java usa un procedimiento llamado **garbage collection** (recolección de basura) para recuperar la memoria ocupada por objetos que ya no son accesibles para el programa. Este trabajo ha pasado a ser responsabilidad del sistema no del programador. Si un objeto tiene varias referencias, y una se pierde, aún puede ser utilizado por el resto. Por eso, un objeto no es candidato a ser basura mientras haya referencias que lo puedan usar.

> **Nota**: *el que los programadores sean responsables del consumo y liberación de la memoria, ha sido una gran fuente de errores y pérdida de recursos en los programas.*

La idea de la recolección de basura viene de los años 1960, pero no se implementó en lenguajes anteriores porque necesita consumir recursos para automatizarla. Pero Java surgió cuando el hardware ya tenía suficiente capacidad para implementarla (para regocijo y alivio de los probrecitos programadores de todo el mundo).

4.10 USAR OBJETOS EN PROGRAMAS.

Hasta ahora nos hemos centrado en el diseño e implementación de nuevas clases. Pero no hay que olvidar que Java tiene un gran número de clases listas para usarlas. Algunas pueden utilizarse para crear objetos directamente y otras para extenderlas (crear nuevas clases a partir de ellas). Un programador experto debe estar familiarizado con las clases preconstruidas y sacarles partido. Veamos algunas.

StringBuilder

Un string se puede reconstruir a partir del operador +, pero no es

eficiente porque implica crear un nuevo string que es una copia del string original (esto tarda demasiado tiempo) con el nuevo carácter añadido:

```
str = str + caracter; // Copiar str y crear un nuevo objeto
```

Si tu programa necesita construir strings grandes añadiendo partes, tendrá que crear muchos objetos para conseguirlo y necesita mucha cantidad de procesamiento (ineficiente). Para solucionarlo está la clase **StringBuilder** del paquete **java.lang**. Ejemplo:

```
import java.lang.StringBuilder;
StringBuilder builder = new StringBuilder();
```

Igual que un String, un StringBuilder contiene una secuencia de caracteres. Sin embargo es posible añadirle nuevos trozos al final sin necesidad de copiar la cadena de letras anterior. Si x es cualquier valor de cualquier tipo y builder es una referencia a un objeto StringBuilder, el comando **builder.append(x)** añade la representación en string de x al final de la cadena de builder de forma eficiente. Puede utilizarse muchas veces hasta acabar de construir builder. Al final, se convierte builder en un String con la función **builder.toString()**. Otros métodos son:

MÉTODO	DESCRIPCIÓN

insert()	inserta una cadena/carácter en la posición indicada. Ej:
	`StringBuilder sb = new StringBuilder("Hola");`
	`sb.insert(0, ' ');  // " Hola"`

delete()	elimina los caracteres que hay entre dos posiciones. Ej:
	`StringBuilder sb = new StringBuilder("Hola");`
	`sb.delete(6, 8);`

replace()	reemplazar los caracteres que hay entre dos

MÉTODO	DESCRIPCIÓN

	posiciones por otros diferentes. Ej:
	`StringBuilder sb = new StringBuilder("Hola");`
	`sb.replace(1,3, "elad"); // "Helada"`

charAt(), indexOf(), lenght()...	Muchos métodos de la clase String

Date

Está en el paquete **java.util**, y se utiliza para representar el tiempo (fechas y horas). Si el objeto se construye sin parámetros el valor inicial representa la fecha y hora actual. También tenemos LocalDateTime, Calendar y clases formateadores de fechas. Ejemplo:

```
import java.util.Date;
System.out.println( new Date() );
```

Random

Permite generar números pseudoaleatorios y está en **java.util.Random**. Ya hemos usado en algunos ejemplos la función Math.random() pero en algunas aplicaciones los valores que genera no son suficientemente aleatorios.

Un objeto de tipo Random puede generar enteros además de números reales. Creamos un objeto generador de números gn de esta forma:

```
Random gn = new Random();
```

si N es un número entero positivo, la llamada **gn.nextInt(N)** devuelve un entero aleatorio entre 0 y N-1. Por ejemplo:

```
import java.util.Random;
```

```
import java.util.Math;

int dado1 = (int)(6 * Math.random()) + 1;
int dado2 = gn.nextInt(6) + 1;  // mejores propiedades aleatorias
```

También tienes otros métodos para generar por ejemplo números aleatorios que sigan una distribución gausiana en vez de uniforme, etc.

Color

Está en el paquete **java.awt**, y representa un color con el que se puede dibujar. Tiene constantes como **Color.RED**. También es posible crear nuevos colores además de los que tiene predefinidos. Tiene muchos constructores. Uno de ellos es **new Color(r,g,b)**,que acepta 3 enteros entre 0 y 255 y representa la cantidad de rojo, verde y azul del color.

Otro constructor es **new Color(r,g,b,t)**, que añade un cuarto parámetro, otro número dentre 0 y 255 que indica la transparencia (a mayor valor más opacidad, a menor valor más transparencia).

Otro constructor tiene un solo entero en el que puedes pasar los componentes con 2 cifras hexadecimales cada una, **new Color(0x8B5413)** crea un color marrón)

Un objeto Color solo tiene unos cuantos métodos getters: **getRed()**, etc. y no tiene setters (es inmutable, como los String).

Arrays

Arrays es una clase estática de **java.util** creada para ofrecer algunos métodos que trabajan sobre Arrays: **Arrays.método();**

MÉTODO	DESCRIPCIÓN
`fill`	Permite rellenar todo o parte de un array unidimensional con un determinado valor.

MÉTODO	DESCRIPCIÓN

	`int a[]= new int[23];`
	`Arrays.fill(a, -1); //Todo el array vale -1`
	`Arrays.fill(a, 5, 8, -1); // De elemento 5 al 7 a -1`

equals	Compara dos arrays y devuelve true si son iguales. Se consideran iguales si son del mismo tipo, tamaño y contienen los mismos valores. A diferencia del operador de igualdad (==), este sí compara el contenido. Ej:
	`int a[] = {2, 3, 4, 5, 6};`
	`int b[] = {2, 3, 4, 5, 6};`
	`int c[] = a;`
	`System.out.println( a == b );  // false`
	`System.out.println( Arrays.equals(a,b) ); // true`
	`System.out.println( a == c);  // true`
	`System.out.println(Arrays.equals(a,c));  // true`

sort	Ordena un array en orden ascendente. Se pueden ordenar todo el array o bien desde un elemento a otro:
	`int x[] = {4,5,2,7,3,8,2,3,9,5};`
	`Arrays.sort(x, 2, 5); // queda {4 5 2 3 7 8 2 3 9 5}`
	`Arrays.sort(x); // Estará completamente ordenado`

binarySearch	Permite buscar un elemento de forma ultrarrápida en un array ordenado (en un array desordenado sus resultados son impredecibles). Devuelve el índice en el que está el elemento. Ej:
	`int x[] = {1,2,3,4,5,6,7,8,9,10,11,12}; Arrays.sort(x);`
	`System.out.println( Arrays.binarySearch(x,8) ); // 7`

copyOf	Crea una copia de un array. Usa dos parámetros, el array a copiar y el tamaño que tendrá el array resultante. Si el tamaño es menor que el array original, sólo obtiene copia de los primeros elementos (tantos

MÉTODO	DESCRIPCIÓN

	como indique el tamaño) y si el tamaño es mayor que el original, devuelve un array en el que los elementos que superan al original se rellenan con el valor por defecto (dependiendo del tipo de datos del array).
	`int a[] = {1,2,3,4,5,6,7,8,9};`
	`int b[] = Arrays.copyOf(a, a.length);`
	`int c[] = Arrays.copyOf(a, 12);`
	`int d[] = Arrays.copyOf(a, 3);  //d es {1,2,3}`

copyOfRange	Como la anterior, sólo que indica con dos números de qué a qué elemento se hace la copia:
	`int a[]= {1,2,3,4,5,6,7,8,9};`
	`int b[]= Arrays.copyOfRange(a, 3, 6); //b es {4,5,6}`

System

El método **System.arraysCopy** de la clase **System** también está relacionado con los arrays, permite copiar un array en otro. Recibe cinco argumentos: el array que se copia, el índice desde que se empieza a copiar en el origen, el array destino, el índice desde el que se copia en el destino, y el tamaño de la copia (número de elementos de la copia). Ejemplo:

```
int uno[] = {1, 1, 2};
int dos[] = {3, 3, 3, 3, 3, 3, 3, 3, 3};
System.arrayCopy(uno, 0, dos, 0, uno.length);
for (int i=0; i <= 8; i++){
   System.out.print( dos[i] + " " );
} //Sale 1 1 2 3 3 3 3 3 3
```

clase String

Ya sabemos declarar variables string a partir de constantes literales o

del operador concatenar (+):

```
String texto1 = “¡Prueba de texto!”;
String texto2;
texto2 = "Este es un texto que ocupa " +
         "varias líneas, no obstante se puede " +  "encadenar";
```

Otras forma de crearlos es usando alguno de sus constructores:

```
char[] palabras = {‘P’,’a’,’l’,'a',’b’,’r’,’a’}; // array de char
String cadena = new String(palabras);
byte[] datos = {97, 98, 99};                     // array de bytes
// Si no indicas el charset, se utiliza ASCII
String codificada = new String (datos, “8859_1”);
```

Lista completa de métodos (ya comentamos algunos, los repetimos, es algo que se utiliza mucho en los programas):

MÉTODO	DESCRIPCIÓN
`char charAt(int i)`	carácter que está en la posición i
`int compareTo(String s)`	Compara las dos cadenas. Devuelve un valor menor que cero si la cadena s es mayor que la original, 0 si son iguales y devuelve un valor mayor que cero si s es menor que la original
`int compareToIgnoreCase( String s)`	Compara dos cadenas, pero no tiene en cuenta si el texto es mayúsculas o no.
`String concat(String s)`	Añade la cadena s a la cadena original.
`String copyValueOf(char[] data)`	Produce un String igual al array de caracteres data.
`boolean endsWith( String s)`	true si la cadena termina con el texto s.
`boolean equals(String s)`	Compara ambas cadenas, true si son iguales.

MÉTODO	DESCRIPCIÓN
`boolean equalsIgnoreCase( String s)`	Compara ambas cadenas sin tener en cuenta mayúsculas y minúsculas.
`byte[] getBytes()`	Devuelve array de bytes de los códigos de cada carácter del String
`void getBytes(int si, int sf, char[] d, int di);`	Almacena contenido de la cadena en el array de caracteres d. Toma los caracteres desde la posición si hasta la posición sf y los copia en el array desde la posición di
`int indexOf(String s)`	Devuelve la posición en la cadena del texto s.
`int indexOf(String s, int pos)`	Devuelve la posición en la cadena del texto s, empezando a buscar desde la posición pos
`int lastIndexOf(String s)`	Devuelve la última posición en la cadena del texto s
`int lastIndexOf(String s, int pos)`	última posición en la cadena del texto s, empezando a buscar desde la posición pos
`int length()`	longitud de la cadena
`boolean matches(String er)`	verdadero si el String cumple la expresión regular
`String replace(char carAnterior, char ncarNuevo)`	cadena idéntica al original pero que ha cambiando los caracteres iguales a carAnterior por carNuevo
`String replaceFirst(String`	Cambia la primera aparición de la cadena str1

MÉTODO	DESCRIPCIÓN
`str1, String str2)`	por la cadena str2
`String replaceAll(String str1, String str2)`	Cambia la todas las apariciones de la cadena uno por la cadena dos
`String startsWith(String s)`	true si la cadena comienza con el texto s.
`String substring(int pos1, int pos2)`	texto que va desde pos1 a pos2.
`char[] toCharArray()`	array de caracteres a partir de la cadena dada
`String toLowerCase()`	Convierte la cadena a minúsculas
`String toLowerCase(Locale local)`	Lo mismo pero siguiendo las instrucciones del argumento local
`String toUpperCase()`	Convierte la cadena a mayúsculas
`String toUpperCase(Locale local)`	Lo mismo pero siguiendo las instrucciones del argumento local
`String trim()`	Elimina los blancos que tenga la cadena tanto por delante como por detrás
`static String valueOf(tipo elemento)`	Devuelve la cadena que representa el valor elemento. Si elemento es booleano, por ejemplo, devolvería una cadena con el valor true.

(Pattern, Matcher) EXPRESIONES REGULARES

Java ofrece las clases **Pattern** y **Matcher** contenidas en el paquete **java.util.regex.***. La clase **Pattern** se utiliza para procesar la

expresión regular y "compilarla" (verificar que es correcta y dejarla lista para su utilización). La clase **Matcher** sirve para comprobar si una cadena cualquiera sigue o no un patrón. Veamoslo con un ejemplo:

```
Pattern p = Pattern.compile("[01]+"); // Valida y procesa patrón
Matcher m = p.matcher("00001010");
if ( m.matches() )
   System.out.println("Si, contiene el patrón");
else
   System.out.println("No, no contiene el patrón");
```

En el ejemplo, el método estático **compile()** de la clase **Pattern** permite crear un patrón, dicho método compila la expresión regular pasada por y genera una instancia de **Pattern** (p en el ejemplo). El patrón p se puede usar varias veces para verificar si una cadena coincide o no con el patrón, dicha comprobación se hace invocando el método **matcher()**, que combina el patrón con la cadena de entrada y genera una instancia de la clase **Matcher** (m en el ejemplo). La clase **Matcher** contiene el resultado de la comprobación y ofrece varios métodos para analizar la forma en la que la cadena ha encajado con un patrón:

- **m.matches()**. true si toda la cadena (de principio a fin) encaja con el patrón o false en caso contrario.
- **m.lookingAt()**. true si el patrón se ha encontrado al principio de la cadena. A diferencia del método matches(), la cadena podrá contener al final caracteres adicionales a los indicados por el patrón, sin que ello suponga un problema.
- **m.find()**. true si el patrón existe en algún lugar de la cadena (no necesariamente toda la cadena debe coincidir con el patrón) y false en caso contrario, pudiendo tener más de una coincidencia. Para obtener la posición exacta donde se ha producido la coincidencia con el patrón podemos usar los

métodos **m.start()** y **m.end()**, para saber la posición inicial y final donde se ha encontrado. Una segunda invocación del método find() irá a la segunda coincidencia (si existe), y así sucesivamente. Podemos reiniciar el método find(), para que vuelva a comenzar por la primera coincidencia, invocando el método **m.reset()**.

Una expresión regular es una expresión textual que utiliza símbolos especiales para hacer búsquedas avanzadas. Las expresiones regulares pueden contener:

- **Caracteres**. Como a, s, ñ, … y se los interpreta tal cual. Si una expresión regular contiene sólo un carácter, matches devolvería verdadero si el texto contiene sólo ese carácter. Si se ponen varios, obliga a que el texto tenga exactamente esos caracteres.
- **Principio y final de línea de entrada**: el acento circunflejo (^) al principio de la expresión regular indica el principio y el dólar ($) el final de la entrada.
- **Caracteres de control:** \\d (dígito, equivale a [0-9]), \\D (cualquier cosa menos un dígito, equivale a [^0-9]), \\s (espacio en blanco, tabulador, salto), \\S (cualquier cosa menos un espacio, \n, …)
- **Caracteres opcionales**. Se ponen entre corchetes. Por ejemplo [abc] significa a, b ó c. Lo que vaya entre corchetes al final representa un solo carácter.
- **Negación de caracteres**. Funciona al revés, no pueden estar los caracteres indicados. Se pone con corchetes dentro de los cuales se pone el carácter circunflejo (^) al principio. [^abc] significa ni a ni b ni c.

- **Rangos**. Se ponen con guiones entre corchetes. Por ejemplo [a-z] significa: cualquier carácter de la 'a' hasta la 'z'.
- **Intersección**. Usa &&. Por ejemplo [a-x&&r-z] significa de la r a la x (intersección de ambas expresiones).
- **Substracción**. Ejemplo [a-x&&[^cde]] significa de la a hasta la x excepto la c, d ó e.
- **Cualquier carácter**. Se hace con el símbolo punto (.)
- **Opcional**. El símbolo ? sirve para indicar que la expresión que le antecede puede aparecer una o ninguna veces. Por ejemplo "a?" indica que puede aparecer la letra a o no.
- **Repetición**. Se usa con el asterisco (*). Indica que la expresión puede repetirse cero o varias veces.
- **Repetición obligada**. Lo hace el signo +. La expresión se repite una o más veces (pero al menos una).
- **Repetición un número exacto de veces**. Un número entre llaves indica las veces que se repite la expresión. Por ejemplo \d{7} significa que el texto tiene que llevar siete números (siete cifras del 0 al 9). Con una coma significa al menos, es decir \d{7,} significa al menos siete veces (podría repetirse más veces). Si aparece un segundo número indica un máximo número de veces \d{7,10} significa de siete a diez veces.
- **Paréntesis**: permiten aplicar el operador de la derecha, no al carácter que resulte de la expresión izquierda, sino a todo lo que resulte dentro del paréntesis. Por ejemplo "#[01]{2,3}" representa las cadenas #00, #11, #000 y #111. Si usamos paréntesis: "(#[01]){2,3}" representa #0#0, #1#1, #0#0#0 y #1#1#1.

Ejemplos:

- [a-z]{1,4}[0-9]+ textos que comienzan por de una a 4 letras seguidas de al menos un dígito numérico o más.
- [XYxy]?[0-9]{1,9}[A-Za-z] representa un DNI o un NIE.

EJEMPLO: Comprobar si el texto que se lee con **JOptionPane** empieza con dos guiones, le siguen tres números y luego una o más letras mayúsculas. De no ser así, el programa vuelve a pedir escribir el texto:

```
import javax.swing.JOptionPane;

public class ExpRegu {

  public static void main(String[] args) {
   String respuesta;
   do {
      respuesta= JoptionPane.showInputDialog("Escribe código: ");
      if( respuesta.matches("--[0-9]{3}[A-Z]+") == false) {
         JoptionPane.showMessageDialog(null, "Texto no casa");
      }
   } while( respuesta.matches("--[0-9]{3}[A-Z]+") == false);
   JOptionPane.showMessageDialog(null,"Expresión correcta!");
  }
}
```

4.10.1 LA CLASE OBJECT

Ya hemos comentado que una de las mayores virtudes de la programación orientado a objetos es poder crear nuevas clases a partir de otras ya existentes. Las subclases (las nuevas) heredan todas las propiedades y el comportamiento de la clase original, pero además puede modificar o añadir nuevas propiedades y comportamiento. En Java todas las clases son subclase de alguna otra, con una única excepción. Si creas una nueva clase, y no extendiende a ninguna otra existente, automáticamente se convierte en subclase de la clase **Object**. Object es la única clase en Java que no es subclase de otra, **es el Adán y la Eva de las clases en Java**.

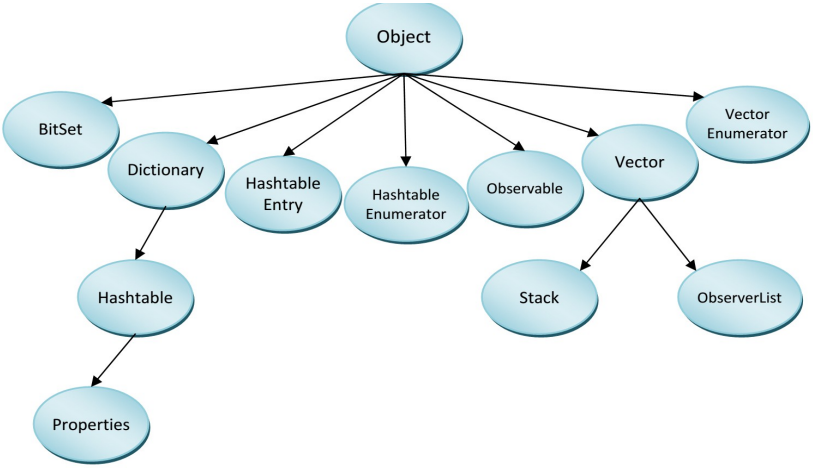


Figura 4: Object está en la raíz de las Jerarquías de clases Java.

La clase **Object** define varios métodos de instancia que son heredados por todo el resto de objetos. Por ejemplo tiene el método de instancia **toString()** que devuelve un String con la representación del objeto. Ya lo has usado de forma implícita cada vez que quería imprimir algún dato no String por pantalla, el objeto se convertía en String gracias a este método.

La versión del método implementada en la clase Object devuelve el nombre de la clase a la que pertenece el objeto concatenado a un número que es el código hash del objeto (no se usa mucho). Cuando crees una nueva clase es conveniente que implementes tu propio método **toString()** y sobreescribas el de la clase Object. Eso te permitirá mostrar por pantalla el objeto. Por ejemplo, vamos a crear el método para la clase Lanza2Dados, que al no extender a otra, es subclase de Object:

```
/**
* Devuelve la representación en texto del lanzamiento
* mostrando el valor de los dos dados
```

```
*/
public String toString() {
   if (dado1 == dado2)
      return "dobles " + dado1;
   else
      return dado1 + " y " + dado2;
}
```

Así podremos hacer cosas como esta:

```
Lanza2Dados dados = new Lanza2dados();
System.out.println( "En los dados salen " + dados );
```

Ejemplos de métodos que otros objetos heredan de esta clase. Algunos de ellos sería conveniente sobreescribirlos siempre (equals y toString), otros según se necesite (clone, hashCode, :

MÉTODO	DESCRIPCIÓN
`Object clone()`	Crea y devuelve una copia del objeto
`boolean equals(Object o2)`	true si los objetos son el mismo (o1 == o2)
`Class<?> getClass()`	la clase en tiempo de ejecución del objeto
`int hashCode()`	un hash del valor del objeto
`String toString()`	representación en String del objeto. Es recomendable que todas las clases lo sobreescriban. Por defecto devuelve: `getClass().getName() + '@' + Integer.toHexString( hashCode() )`
`void notify()`	Si hay threads bloqueadas esperando a poder usar el objeto, una de ellas se desbloquea.

MÉTODO	DESCRIPCIÓN
`void notifyAll()`	Se desbloquean todos los threads bloqueados en el objeto.
`void wait()`	El thread actual se bloquea en el objeto
`void wait(Long ms)`	El thread actual se bloquea en el objeto durante ms milisegundos como máximo
`void wait(Long ms, int ns)`	El thread se bloquea como máximo ms milisegundos y ns nanosegundos

4.10.2 EJEMPLO DE COMO ESCRIBIR CLASES.

En POO la metodología de la programación consiste en crear clases y objetos en el programa que imiten el estado y el comportamiento de los cosas que existen en el mundo real y que tu programa va a utilizar.

Imagina que vamos a programar juegos de cartas y queremos ir implementando clases que puedan ser reutilizables en varios proyectos. Los juegos de cartas se juegan con una baraja que contiene cartas.

En un típico juego de cartas, el mazo de cartas se baraja (se mezclan y se desordenan las cartas) y a cada jugador se le reparte cierta cantidad de cartas del mazo. Ese reparto de cartas es una mano. Hay juegos donde a la mano se le quitan y añaden cartas. Las jugadas del juego se ganan o pierden según los valores de las cartas (as, dos, ... rey) y del palo (espadas, copas, oros y bastos).

Si pensamos en esta descripción, vemos que hay muchas posibles clases: juego, jugador, mano, mazo, carta, valor, palo ... Aunque el valor y el palo son simples valores que describen una carta. Vamos a centranos en las que podrían ser más reusables: carta, mazo y mano.

Si nos fijamos en los verbos de la descripción de un juego de cartas, extraeremos el comportamiento que debemos proporcionar a las clases (barajar, sacar carta del mazo, etc.). Así que un mazo tiene 2 métodos posibles: **barajar()** y **sacarCarta()**. Las cartas pueden añadirse y eliminarse de las manos, pues una mano tendrá: **insertaCarta()** y **quitaCarta()**. Las cartas son cosas pasivas, pero al menos necesitaremos saber su valor y su palo. Seguiremos después.

Vamos a centranos en el mazo. Necesita un constructor **Mazo()** que defina las cartas que contiene. La operación **baraja()** no da problemas. La operación **sacaCarta()** podría fallar si el mazo ya no tiene cartas. Habrá que crear una excepción en ese caso. Pero, ¿Cómo puede saber el resto del mundo que no quedan cartas en el mazo? Bueno, habrá que proporcionar otro método como **quedan()** que nos digan las cartas que tiene el mazo todavía o **isVacio()**. Así que ya tenemos un posible esqueleto de la clase **Mazo**:

```
/**
* Constructor. Crea un mazo ordenado de cartas
*/
public Mazo()
/**
* Desordena de forma aleatoria las cartas del mazo
*/
public void baraja()
/**
* Indica cuantas cartas quedan en el mazo
*/
public int quedan()
/**
* Quita una carta del mazo y la devuelve.
* @throws IllegalStateException si no quedan cartas.
*/
public Carta sacaCarta()
```

Esto ha sido un ejercicio de diseño, no nos hemos planteado cómo

implementar cada operación, solamente hemos pensado las operaciones que debe tener la clase.

Podemos hacer algo similar para la clase Mano.

- Cuando se crea, aún no tendrá cartas. Necesita una operación **insertaCarta()**. Este método tendrá como parámetro la carta que se añade a la mano de un jugador.
- Cuando el jugador se deshaga de una carta de su mano, el método **dejaCarta()** tendrá de parámetro la carta que elimina de la mano. Podrías indicar deja el as de espadas, o deja la tercera carta de la mano. Como no sabemos qué es mejor, lo ideal es ofrecer ambas posibilidades. Así que sobrecargamos el método cambiando los parámetros.
- Puede ser interesante poder conocer cuantas cartas tiene la mano, así que definimos el método **cuentaCartas()**.
- Cuando juegas a cartas, normalmente un jugador se ordena las cartas de alguna forma, así que definiremos métodos de instancia que lo hagan, **ordenaCartas()**. La especificación será algo así:

```
/**
* Constructor. Crea una mano sin cartas.
*/
public Mano()
/**
* Descartar todas las cartas de la mano.
*/
public void vacia()
/**
* Añadir la carta c a la mano. c no puedde ser null.
* @throws NullPointerException si c es null.
*/
public void insertaCarta(Carta c)
/**
* Si la carta indicada está en la mano, se quita.
```

```
*/
public void dejaCarta(Carta c)
/**
* Elimina la carta de la posición indicada de la mano.
* Se numeran las posiciones desde 0.
* @throws IllegalArgumentException si la posición no existe.
*/
public void dejaCarta(int posicion)
/**
* Devuelve el nº de cartas en la mano.
*/
public int cuentaCartas()
/**
* Obtener la carta de una posición en la mano.
* Las posiciones se numeran comenzando en 0.
* @throws IllegalArgumentException si la posicion no existe.
*/
public Carta dameCarta(int posicion)
/**
* Ordena las cartas de la mano agrupando por palo
* y luego por valor
*/
public void ordenaPorPalo()
/**
* Ordena las cartas de la mano por valor y luego por palo.
*/
public void ordenaPorValor()
```

En cuanto a la clase **Carta**:

- Tendrá un constructor donde se indique el valor y el palo. Hay 4 palos que se pueden representar con los enteros 0, 1, 2 y 3. Pero así va a ser lioso acordarse de como se han asociado los números a los palos, así que podemos usar constantes sombólicas (**Carta.ESPADAS** representa el palo espadas por ejemplo y se declaran públicas, estáticas y final). También se podría usar un tipo enumerado, pero lo dejaremos con constantes simbólicas. Los posibles valores de una carta son números del 1 al 10, el 1 son los ases, los 10 son las sotas, los

onces los caballos y los doces los reyes. Las figuras (valores 1, 10, 11 y 12) con constantes simbólicas. Así que un objeto carta puede construirse así:

```
carta1 = new Carta( Carta.AS, Carta.ESPADAS ); // As de espadas
carta2 = new Carta( 3, Carta.BASTOS );         // 3 de bastos
carta3 = new Carta( v, s ); // OK, si v y s son expresiones int
```

- Un objeto **Carta** necesita variables de instancia que contengan su valor y su palo. Mejor privadas para que no se cambien desde fuera de la clase. Así que necesita los getters **getPalo()** y **getValor()**. Las variables de instancia las inicializa el constructor, y no hay motivo para que cambien (modificador **final**), así que son objetos immutables.

Por último para facilitar la interacción con los usuarios se definen métodos **getPaloString()**, **getValorString()** y **toString()** para que se puedan usar sentencias como:

```
System.out.println( "Su carta es " + carta );
```

que es equivalente a:

```
System.out.println( "Su carta es " + carta.toString() ) ;
```

Así que la implementación de la clase Carta podría ser esta:

```
package naipes;
/**
* Un objeto de tipoCarta representa una Carta de un
* mazo de la baraja española. La carta tiene un palo
* que puede ser espadas, bastos, copas y oros y un valor
* que va de 1 a 12: as, 2, 3, 4, 5, 6, 7,
* 8, 9, sota, caballo, rey. El as es el menor valor.
*/
public class Cata {
   // Palos
   public final static int ESPADAS = 0;
   public final static int BASTOS = 1;
```

```
      public final static int COPAS = 2;
      public final static int OROS = 3;
      public final static int COMODIN = 9;
      public final static int AS = 1;
      // Valores
      public final static int SOTA = 10;
      public final static int CABALLO = 11;
      public final static int REY = 12;
   /**
   * El palo debe ser ESPADAS, BASTOS, COPAS, OROS o COMODIN
   */
   private final int palo;
   /**
   * El valor de la carta de 1 a 12
   */
   private final int valor;
   /**
   * Crea una carta con datos fijos
   */
   public Carta() {
      palo = COPAS;
      valor = 1;
   }
   /**
   * Crea una carta indicando palo y valor
   * @param elValor entre 1 y 12
   * @param elPalo puede ser
   * Carta.ESPADAS, Carta.BASTOS, Carta.COPAS o Carta.OROS.
   * @throws IllegalArgumentException si valores fuera de rango
   */
   public Carta(int elValor, int elPalo) {
      if (elPalo != ESPADAS && elPalo != BASTOS && elPalo != COPAS
          && elPalo != OROS)
         throw new IllegalArgumentException("Palo ilegal");
      if (elValor < 1 || elValor > 12)
         throw new IllegalArgumentException("Valor ilegal");
      valor = elValor;
      palo = elPalo;
   }
   /**
   * Devuelve el palo de la carta.
   * @returns el palo que es  Carta.ESPADAS, Carta.BASTOS,
   * Carta.COPAS o Carta.OROS.
```

```
*/
public int getPalo() { return palo; }
/**
* Devuelve el valor.
* @return el valor, un número entre 1 y 12
*/
public int getValor() { return valor; }
/**
* Devuelve un String con el  palo de la carta.
* @return uno de los strings "Espadas", "Bastos", "Copas", "Oros"
*/
public String getPaloString() {
   switch ( palo ) {
   case ESPADAS: return "Espadas";
   case BASTOS:  return "Bastos";
   case COPAS:   return "Copas";
   case OROS:    return "Oros";
   default: return "Comodín";
   }
}
/**
* Devuelve una representación String del valor.
* @return un strings "As", "2",
* "3", ..., "Sota", "Caballo", "Rey"
*/
public String getValorString() {
   if ( palo == COMODIN )
      return "" + valor;
   else {
      switch ( valor ) {
      case 1: return "As";
      case 2: return "2";
      case 3: return "3";
      case 4: return "4";
      case 5: return "5";
      case 6: return "6";
      case 7: return "7";
      case 8: return "8";
      case 9: return "9";
      case 10: return "Sota";
      case 11: return "Caballo";
      default: return "Rey";
      }
```

```
      } // fin else
   }
   /**
   * Devuelve un string de la carta, valor y palo
   */
   public String toString() {
      if (palo == COMODIN) {
         if (valor == 1)
            return "Joker";
         else
            return "Joker #" + valor;
      }
      else
         return getValorString() + " de " + getPaloString();
   }
   } // fin clase
```

EJEMPLO: Implementar un sencillo juego que use las clases anteriores. Se baraja el mazo de cartas y se saca una carta mostrándola al usuario. El usuario predice si la siguiente carta es mayor o menor que la actual. Si acierta, la siguiente carta del mazo se convierte en la actual y debe hacer otra predicción. Se continúa hasta que comete un fallo. La puntuación es el número de predicciones que alcanza el jugador.

```
   /**
   * Implementa el juego de cartas AltoBajo.
   */
   import java.util.Scanner;

   public class AltoBajo {

      static Scanner teclado = new Scanner( System.in );

      public static void main(String[] args) {
         System.out.println("Es un sencillo juego de cartas,");
         System.out.println("AltoBajo. Se saca una carta del mazo.");
         System.out.println("Debes predecir si la siguiente es ");
         System.out.println("mayor o menor. Tu puntuación es ");
         System.out.println("el número de aciertos antes de fallar");
         System.out.println();
```

```
      int partidas = 0;    // Nº de partidas jugadas
      int sumaPuntos = 0; // La suma de puntuaciones
      double mediaPuntos; // Media de las puntuaciones
      boolean jugarOtra;  // Respuesta del usuario
      do {
         int puntos;      // Puntos en este juego
         puntos = jugar();   // Jugar una vez
         sumaPuntos += puntos;
         partidas++;
         System.out.print("Juega otra vez? ");
         juegaOtra = teclado.nextBoolean();
      } while (jugarOtra);
      mediaPuntos = ((double)sumaPuntos) / partidas;
      System.out.println();
      System.out.println("Ha jugado " + partidas + " veces.");
      System.out.printf("Puntuación media %1.3f.\n",mediaPuntos);
   } // fin main()
   /**
   * Jugar un juego y devolver puntos obtenidos
   */
   private static int jugar() {
      Mazo m = new Mazo(); // Crear mazo de cartas
      Carta actual;        // La carta que ve el usuario
      Carta siguiente;     // La sig. Carta del mazo
      int aciertos;        // Nº de predicciones Ok
      char prediccion;     // 'A' más alta, 'B' más baja
      m.baraja();          // Desordena aleatoriamente
      // Comienza el juego
      aciertos = 0;
      actual = m.dameCarta();
      System.out.println("La primera carta es " + actual);
      while(true) {        // Repetir hasta un fallo
         /* Obtener predicción del usuario, 'A'/'B'/'a'/'b' */
         System.out.print("Siguiente carta Alta(A) o Baja (B)? ");
         do {
            prediccion = teclado.next();
            prediccion = Character.toUpperCase(prediccion);
            if (prediccion != 'A' && prediccion != 'B')
              System.out.print("Responda con A o B ");
         } while (prediccion != 'A' && prediccion != 'B');
         /* Obtener siguiente carta */
         siguiente = m.dameCarta();
         System.out.println("La siguiente carta es " + siguiente);
```

```
            /* comprueba la predicción */
            if (siguiente.getValor() == actual.getValor()) {
               System.out.println("Igual valor, pierdes.");
               break;  // fin del juego
            }
            else if (siguiente.getValor() > actual.getValor()) {
               if (prediccion == 'A') {
                  System.out.println("Tu predicción es correcta.");
                  aciertos++;
               }
               else {
                  System.out.println("Tu predicción es incorrecta");
                  break; // fin del juego
               }
            }
            else {         // la siguiente es menor
               if (prediccion == 'B') {
                  System.out.println("Tu predicción fue correcta.");
                  aciertos++;
               }
               else {
                  System.out.println("Predicción incorrecta.");
                  break; // fin del juego
               }
            }
            /* Hay que convertir la siguiente en la actual y
               comprobar que quedan cartas en el mazo */
            actual = siguiente;
            System.out.println();
            System.out.println("La carta es " + actual);
         } // fin del buclle
         System.out.println();
         System.out.println("El juego se ha acabado.");
         System.out.println("Tienes " + aciertos + " puntos.");
         System.out.println();
         return aciertos;
      } // fin juega()
   } // fin clase
```

EJERCICIO: Completa lel código de las clases que faltan (Mazo y Mano) y haz que este juego pueda funcionar.

4.11. EJERCICIOS.

E1. TEST

T1. Una caja negra tiene una interfaz y una implementación. Explica el significado de estos dos elementos.

T2. Una subrutina se dice que es un contrato ¿En qué consiste este contrato? (cuando quieras usarla qué debes tener presente en ese contrato). Este contrato tiene un aspecto sintáctico y otro semántico. ¿En qué consiste cada uno?

T3. Explica brevemente como las subrutinas ayudan a diseñar programas de forma top-down.

T4. ¿Para qué sirven los parámetros de una subrutina? ¿Qué diferencia hay entre los parámetros actuales y formales?

T5. ¿Qué es una API? Da un ejemplo en Java.

T6. Escribe una subrutina llamada **estrellas** que imprima una línea de asteriscos. La cantidad de estrellas será un parámetro. ¿Cómo se escribe una llamada a la subrutina desde la misma clase?

T7. Escribe una rutina **main()** que use la subrutina **estrellas** del ejercicio T6 para que muestre 10 líneas, la primera con una estrella, la segunda con dos estrellas, etc.

```
*
**
***
****
*****
******
*******
********
```

```
*********
**********
```

T8. Escribe una función llamada **cuentaLetras()** que tenga de parámetros un String y un char. Devuelve el número de veces que aparece el carácter en el String.

T9. Escribe una subrutina de 3 parámetros que sean números enteros. La subrutina debe averigual cuál de ellos es el menor y devolverlo.

T10. Escribe una subrutina que encuentre y devuelva la media de los primeros n elementos de un array de tipo double. El array y el número n son parámetros.

T11. Explica lo que hace la siguiente función:

```
static int[] stripZeros( int[] lista ) {
   int cont = 0;
   for (int i = 0; i < lista.length; i++) {
      if ( lista[i] != 0 )
         cont++;
      }
   int[] nuevaLista;
   nuevaLista = new int[cont];
   int j = 0;
   for (int i = 0; i < lista.length; j++) {
      if ( lista[i] != 0 ) {
         nuevaLista[j] = lista[i];
         j++;
      }
   }
   return nuevaLista;
}
```

T12. Complete las siguientes oraciones:

a) Un método se invoca con una _____.

b) A una variable que se conoce sólo dentro del método en el que está declarada, se le llama _____.

c) La instrucción _____ en un método llamado puede usarse para devolver el valor de una expresión al método que hizo la llamada.

d) La palabra clave _____ indica que un método no devuelve ningún valor.

e) Las tres formas de regresar el control de un método llamado a un solicitante son _____, _____ y _____.

f) El _____ de una declaración es la porción del programa que puede hacer referencia por su nombre a la entidad en la declaración.

g) Es posible tener varios métodos con el mismo nombre, en donde cada uno opere con distintos tipos o números de argumentos. A esta característica se le llama _____ de métodos.

T13. Escriba el encabezado para cada uno de los siguientes métodos:

a) El método hipotenusa, que toma dos argumentos de punto flotante con doble precisión, llamados lado1 y lado2, y que devuelve un resultado de punto flotante, con doble precisión.

b) El método menor, que toma tres enteros (x, y, z) y devuelve un entero.

c) El método instrucciones, que no toma argumentos y no devuelve ningún valor.

d) El método intAFloat, que toma un argumento entero llamado numero y devuelve un resultado de punto flotante (float).

T14. Encuentre el error en cada uno de los siguientes segmentos de programas. Explique cómo se puede corregir.

a)

```
void g() {
   System.out.println("Dentro del método g");
```

```
      void h() {
         System.out.println("Dentro del método h");
      }
   }
```

b)

```
   int suma(int x, int y) {
      int resultado;
      resultado = x + y;
   }
```

c)

```
   void f(float a); {
      float a;
      System.out.println(a);
   }
```

d)

```
   void producto() {
      int a = 6, b = 5, c = 4, resultado;
      resultado = a * b * c;
      System.out.printf("El resultado es %d\n", resultado);
      return resultado;
   }
```

e)

```
   void tercio(double a, double b) {
      a = 0.33;
      return a * b;
   }
```

T15. Declare el método **volumenEsfera()** para calcular y mostrar el volumen de la esfera. Utilice la siguiente instrucción para calcular el volumen:

```
double volumen = (4.0 / 3.0) * Math.PI * Math.pow(radio, 3);
```

Escriba una aplicación en Java que pida al usuario el radio double de una esfera, que llame a volumenEsfera() para calcular el volumen y muestre el resultado en pantalla.

E2. Queremos hacer una clase que encapsule el nombre y la edad de un empleado que debe ser mayor o igual que 0. Además necesitamos conseguir que la edad sea obligatoria. Analiza si la siguiente definición de la clase y descubre y soluciona los problemas que pueda tener (A parte de errores léxicos, quizás proporcionando un constructor y cambiando algo...).

```
// Fichero Empleado.java
Public class empleado {
       String nombre;
       int edad;
   }

// Fichero PruebaEmpleados.java
public class PruebaEmpleados {
       public static void main(String[] args) {
              Empleado e1 = new Empleado();
              System.out.println( e1.edad );
              e1.edad = 14;
       }
}
```

E3. A la clase Empleados del ejercicio E2 queremos añadirle un método estático (ya que hace lo mismo para todos los objetos) que devuelva el nombre del empleado, algo así:

```
public static String getNombre() {  return this.nombre;  }
```

Analiza la solución propuesta y soluciona el problema.

E4. A la clase Empleados del ejercicio E2 queremos añadirle el dato sueldo y un password común para que lo usen en el departamento de recursos humanos. Necesitamos poder consultar y modificar todos los datos de un empleado (salvo los passwords) desde fuera de la clase, y también modificarlos, pero seguir asegurando que la edad de cada uno

es mayor o igual a 18 y que el sueldo solamente lo podrán consultar/cambiar si conocen un password común que compartan todos los objetos. Piensa en una forma de conseguir todo esto.

E5. Necesitamos una clase para manejar puntos en 2 dimensiones. Cada punto queda definido por su coordenada x y por su coordenada y. Creamos la clase con un constructor al que pasamos los dos valores. Analiza el código y encuentra y soluciona los errores.

```
// Fichero Punto2D.java
Public class empleado {
       public double x, y;

       public void Punto2D(double x, double y) {
              x = x;
              y = y;
       }
   }

// Fichero PruebaEmpleados.java
public class PruebaPuntos {
       public static void main(String[] args) {
              Punto2D p1 = new Punto2D(3, 2.5);
              System.out.println( p1.x, p2.y );
       }
}
```

E6. Queremos que al usar el constructor por defecto (sin argumentos) se cree el punto (0,0). Además preferimos no añadir más sentencias innecesarias a las que necesitemos. ¿Qué tal te parecen estas soluciones sabiendo que `public Punto2D() { x = 0; y = 0; }` está descartada porque añadimos sentencias?

a) No hacemos nada y cuando se use new Punto2D(); Java usará el constructor de Object, que crea el objeto e inicializa las variables miembro x, y a 0 al no ser locales. Razona la respuesta

y luego la confirmas escribiendo código que lo pruebe.

b) `public Punto2D Punto2D() { return new Punto2D(0, 0); }`

c) `public Punto2D() { Punto2D this = new Punto2D(0,0); }`

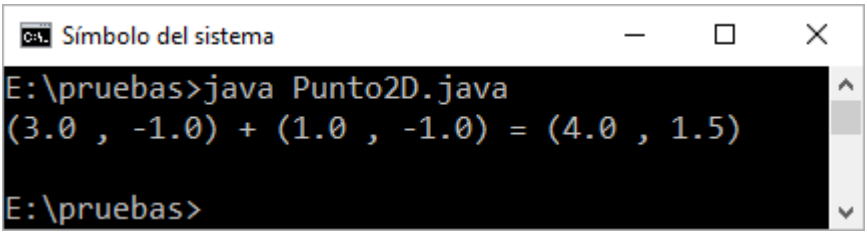
Si no te cuadra ninguna de las anteriores, piensa como hacer el constructor usando el que ya tienes.

E7. Queremos añadir a la clase `Punto2D` getters y setters y una operación suma de forma que reciba de argumento un `Punto2D` y devuelva un nuevo `Punto2D` con la suma de ambos:

(x1, y1) + (x2, y2) = (x1 + x2, y1 + y2)

Crea un método main para probar la clase y completa las siguientes sentencias hasta conseguir una salida como esta:

```
Punto2D p1 = new Punto2D(3, 2.5);
Punto2D p2 = new Punto2D(1, -1);
Punto2D p3 = p1.suma( p2 );
System.out.println( /* ... completa ... */ );
```



E8. Nos damos cuenta que si queremos imprimir muchos puntos, ahora mismo es un poco lioso. ¿Cómo conseguir que al pasar un punto se imprima "(x, y)"? Sabes que `Object`, como buen padre, presta sus métodos incluido `toString()` a todos sus hijos, incluido Punto2D. Por

tanto, cambiamos la última línea del código de prueba anterior por:

```
System.out.println( p1 + " + " + p2 + " = " + p3);
```

Si no obtienes la salida que querías, quizás tengas que sobreescribir (sustituir, override en inglés) el método `toString()` de `Object`.

E9. Ahora necesitas saber si dos Punto2D son iguales para poder compararlos:

```
p2.setX( p1.getX() );
p2.setY( p1.getY() );
System.out.println(p1 + " es igual a + " p2 + "? " + (p1 == p2) );
```

Como son objetos, supongo que a estas alturas ni se te pasará por la cabeza usar el operador == de arriba (solamente te serviría para saber si son el mismo objeto, en cuyo caso sí serían iguales).

Sobreescribe el método equals() de Object que permita comparar Puntos2D con el resto de objetos.

E10. Utiliza la anotación **@Override** delante del método **equals()** de Punto2D para comprobar si has realizado bien la sustitución del método del padre, o bien si al equivocarte lo que has hecho es sobrecargar el nombre del método (ligadura dinámica) en vez de sobreescribirlo. Si lo has sobreescrito mal, debes corregirlo. Queremos sustituirlo, no sobrecargar el método **equals()**. Prueba el método corregido con el código de prueba correcto.

E11. Copia el main de la clase Punto2D a una clase pública llamada U04EA11 y haz que Punto2D no sea pública. Guarda los cambios en el fichero .java con el mismo nombre que la clase pública. Añade estas dos nuevas líneas de código a la clase de prueba de Punto2D:

```
// Sentencia 1
 if( p1.x >= 0  && p2.y >= 0) {
        System.out.println( p1 + " en primer cuadrante"  );
    }
// Sentencia 2 equivalente
 if( p1.getX() >= 0  && p2.getY() >= 0) {
        System.out.println( p1 + " en primer cuadrante"  );
    }
```

Ahora hemos descubierto que sería ventajoso usar internamente un array para almacenar las coordenadas x e y de cada Punto2D, la x en la posición 0, la y en la posición 1 del array. Así podríamos fácilmente crear puntos 3D, 4D, 1D... Cambia el código de la clase Punto2D y sustituye la línea: **`public double x, y;`** por **`double[] c;`**
Adapta el resto de código de la clase a esta nueva definición y responde (de forma razonada y probando si es necesario) a estas preguntas:

- ¿A cuál de las dos líneas afectará el cambio?
- ¿Es bueno usar los getters y setters de los objetos una clase en vez de acceder directamente a sus variables miembro? Respnde V (verdadero) y F (falso):
  - Así la clase puede implementar reglas de negocio (restricciones lógicas) y forzar su cumplimiento: ____.
  - Así los cambios internos en cada clase no afectan al código que los usa si la interfaz no cambia: ___.
  - Asegurar los dos puntos anteriores exige que la clase oculte (proteja al máximo) las variables miembro con los modificadores de acceso adecuados): ___.
  - Lo razonable (salvo que existan causas que lo juestifiquen) es declarar variables miembro como private y métodos de uso externo como public: __.

E12. Capitalizar un string significa convertir las primeras letras de

cada palabra del string a mayúscula (si no lo está ya). Por ejemplo, "Ha llegado el momento de la verdad" se convierge en "Ha Llegado El Momento De La Verdad". Haz una subrutina que capitalice un string que se le pasa como parámetro (si observas capitaliza la primera letra de cada frase/primera de cada palabra). Prueba la subrutina en una rutina main() que:

> **Nota**: usa la clase StringBuilder para hacerlo de forma eficiente y las funciones `Character.isLetter(char)` y `Character.toUpperCase(char)`.

a) Obtenga la cadena preguntándola al usuario.

```
Símbolo del sistema
E:\pruebas>java U04EA12A.java
 Teclee una frase: el amazonas.el mayor rio del mundo
Capitalizada: El Amazonas.El Mayor Rio Del Mundo

E:\pruebas>_
```

b) Obtenga la cadena como parámetro del programa.

```
Símbolo del sistema
E:\pruebas>java U04EA12B.java
 Debe pasar la frase a capitalizar

E:\pruebas>java U04EA12B.java vini, vidi, vinchi
Capitalizada: Vini, Vidi, Vinchi

E:\pruebas>_
```

c) Si no tiene parámetro, pregunte al usuario la cadena, si tiene parámetro, lo capitalice.

E13. Los dígitos hexadecimales son los decimales ('0' a '9') y las letras de la 'A' a la 'F'. En el sistema de numeración hexadecimal los dígitos representan valores de 0 a 15. Escribe una función llamada **hexValor()** que usando una sentencia **switch**, encuentre el valor hexadecimal de un carácter que se pasa como parámetro (debe dar igual mayúsculas o minúsculas, 'f' y 'F' valen 15). Si el carácter que se pasa no está entre '0' y 'F' devuelve -1.
Una cifra en hexadecimal es una cadena con dígitos hexadecimales, como 34A7, ff8, 174204 o FADE. Si s es una cadena que contiene un número expresado en hexadecimal, entences su valor en decimal se puede calcular así:

```
valor = 0;
for ( i = 0; i < s.length(); i++ )
   valor = valor * 16 + hexValor( s.charAt(i) );
```

Haz un programa que acepte como parámetro una cadena hexadecimal, y si no se la pasan que la pida al usuario y muestre el valor decimal si es válido o un mensaje de error si tiene caracteres no hexadecimales.

E14. Escribe una función que simule el lanzamiento de un par de dados hasta que salga el número que pasas como parámetro (debe estar entre 2 y 12). La función devuelve la cantidad de tiradas que ha costado sacar ese número.

a) Usa una aserción para controlar la precondición y la postcondición (lo que haya) con un mensaje personalizado indicando la función donde se produce el error en el mensaje.

b) Ahora sustituye la aserción por una una excepción de parámetros para asegurarte de que la función funcionará.

E15. Escribe un método que acepte un número arbitrario de

argumentos enteros, (como mínimo 2) y devuelva el máximo.

E16. Para redondear números a lugares decimales específicos, use una instrucción como la siguiente:

```
y = Math.floor(x * 10 + 0.5) / 10;
```

que redondea x en la posición de las décimas (es decir, la primera posición a la derecha del punto decimal), o:

```
y = Math.floor(x * 100 + 0.5) / 100;
```

que redondea x en la posición de las centésimas (es decir, la segunda posición a la derecha del punto decimal). Escriba una aplicación que defina cuatro métodos para redondear un número x en varias formas:

redondearA(x, decimales)

que redondea x en la posición de las centésimas (es decir, la segunda posición a la derecha del punto decimal). Escriba una aplicación que defina un método para redondear un número x de varias formas:

E17. Se dice que un número entero es un *número perfecto* si sus factores, incluyendo el 1 (pero no el propio número), al sumarse dan como resultado el número entero. Por ejemplo, 6 es un número perfecto ya que 6 = 1 + 2 + 3.

Escriba un método llamado **esPerfecto()** que determine si el parámetro **numero** es un número perfecto. Use este método en una aplicación que muestre todos los números perfectos entre 1 y 1,000. Muestre en pantalla los factores de cada número perfecto para confirmar que el número sea realmente perfecto. Ponga a prueba el poder de su computadora: evalué números más grandes que 1,000. Muestre los resultados.

E18. Se dice que un entero positivo es *primo* si puede dividirse de forma exacta solamente por 1 y por sí mismo. Por ejemplo: 2, 3, 5 y 7 son primos, pero 4, 6, 8 y 9 no. Por definición, el número 1 no es primo.

a) Escriba un método que determine si un número es primo.

b) Use este método en una aplicación que determine y muestre en pantalla todos los números primos menores que 10,000. ¿Cuántos números hasta 10,000 tiene que probar para asegurarse de encontrar todos los números primos?

c) Al principio podría pensarse que *n*/2 es el límite superior para evaluar si un número *n* es primo, pero sólo es necesario ir hasta la raíz cuadrada de *n*. Vuelva a escribir el programa y ejecútelo de ambas formas.

E19. Intente aplicar el diseño top-down a este problema e identificar las subrutinas que sería conveniente implementar. Escriba una aplicación que muestre un menú con 3 opciones:

```
        Menu
        ====

1. Lanzar moneda.
2. Ver resultados.
3. Salir.

  Escoja una opción (1-3):
```

El programa muestra repetidamente el menú hasta que la opción elegida sea alguna de la 1 a la 3. Si es la '3', el programa acaba. Si es la '1' (Lanzar moneda), de forma aleatoria muestra si ha salido cara o cruz y vuelve a mostrar el menú. Cada vez que se lanza la moneda, cuenta el número de veces que aparezca cada uno de los lados de la moneda. Si elige la opción '2' se muestra en pantalla qué porcentaje de

caras y de cruces han salido y vuelve a mostrar el menú.

E20. Crea una clase llamada **CuentaBancaria** que tendrá los siguientes atributos:

- Titular y saldo (puede tener decimales). El titular será obligatorio y la cantidad es opcional.
- Crea dos constructores que cumplan lo anterior.
- Crea sus métodos get, set y toString.
- Tendrá dos métodos especiales:
  - ingresar(double cantidad): se ingresa una cantidad a la cuenta, si la cantidad introducida es negativa, no se hará nada.
  - retirar(double cantidad): se retira una cantidad de la cuenta, si restando la cantidad actual a la que nos pasan es negativa, se lanza una excepción.

E21. Haz una clase llamada **Cliente** que utiliza una clínica de una dietista y que siga las siguientes condiciones:

- Sus atributos son: nombre, edad, numPaciente, sexo (H hombre, M mujer), peso y altura. No queremos que se accedan directamente a ellos, así que piensa que modicador de acceso es el más adecuado. Por defecto, todos los atributos menos el numPaciente serán valores por defecto según su tipo. Sexo sera hombre por defecto, usa una constante para ello.
- Se necesitan varios constructores: Un constructor por defecto. Un constructor con el nombre, edad y sexo. Un constructor con todos los atributos.
- Los métodos que se implementaran son:
  - **calcularIMC()**: calcula si la persona esta en su peso ideal (peso en kg/(altura^2 en m)), si esta fórmula devuelve un

valor menor que 20, la función devuelve un -1, si devuelve un número entre 20 y 25 (incluidos), signica que esta por debajo de su peso ideal y la función devuelve un 0 y si devuelve un valor mayor que 25 signica que está por encima de su peso ideal y devuelve un +1.

- **MayorDeEdad()**: indica si es mayor de edad, devuelve un booleano.
- **comprobarSexo(char sexo)**: comprueba que el sexo introducido es correcto. Si no es correcto, sera H. No será visible al exterior.
- **toString()**: devuelve toda la información del objeto.
- **generaNumPaciente()**: genera un número de paciente usando un valor estático que cuenta los objetos de la clase, inicialmente a 0. Genera a partir de este su número de paciente. Este método sera invocado cuando se construya el objeto. No será visible al exterior.
- Métodos set de cada parámetro, excepto de numPaciente.

Ahora, crea una clase ejecutable que haga lo siguiente: Pide por teclado el nombre, la edad, sexo, peso y altura. Crea 3 objetos de la clase anterior, el primer objeto obtendrá las anteriores variables pedidas por teclado, el segundo objeto obtendrá todos los anteriores menos el peso y la altura y el último por defecto, para este último utiliza los métodos set para darle a los atributos un valor.
Para cada objeto, deberá comprobar si esta en su peso ideal, tiene sobrepeso o por debajo de su peso ideal con un mensaje. Indicar para cada objeto si es mayor de edad. Por último, mostrar la información de cada objeto. Puedes usar métodos en la clase ejecutable.

E22. Vamos a realizar una clase llamada ConjuntoEnteros que define el

comportamiento de un conjunto de números enteros. Sus objetos deben ser immutables.

- Puede tener de 0 a infinitos elementos.
- No tiene elementos repetidos.

Tienen:

- Un nombre
- Un array de elementos enteros.
- Constructores: por defecto y al que s ele pasa un array.
- Operaciones públicas: boolean esVacio(), int cardinal(), void insertaElemento(int), borraElemento(int), boolean pertenece(int), ConjuntoEntero union(ConjuntoEntero c).
- Implementa: toString() y equals().